卒　業　論　文

量子 LLM 支援プログラミングにおける
失敗類型と介入カタログの実証研究

2023TC011 伊藤 悠真

指導教員　横山 哲郎

2027 年 1 月

南山大学 理工学部 電子情報学科

要約

LLM による量子回路コード生成では，API の誤用や回路構築の失敗に加え，実行できても期待した出力を返さない失敗が生じる．本研究では，Qiskit を用いた量子回路コード生成において，初期失敗に対する修正介入と独立再生成を比較し，追加成功数，成功対象および計算コストを分析した．自己見直し R，実行結果提示 F，説明・原因分析を促す自己デバッグ D と，独立再生成 B4 を，同じ初期コードを起点として比較した．生成前注意事項 P は独立生成として扱った．全 1,000 初期組の最終 L3 成功率は B45.5%，P47.3%，R55.1%，F63.8%，D63.4%，B4 67.1%であった．
対象は 5 モデル，10 タスク，seed 0〜19 である．モデル・タスク・seed の組合せを初期組と呼び，B/R/F/D/P の 1,000 初期組・5,000 条件記録を対象とし，B4 の追加実験を別に集計する．検証済みの生成履歴の前半部分と，必要な部分追試による後半部分を組み合わせ，現行評価器の停止・修正規則に従って到達可能な生成履歴（以下，到達履歴）を採用した．本研究における最終評価水準（L3）は，指定出力・構造要件・共通 system の回路内測定禁止を含む成功基準である．
初期失敗 545 組の救済は R96 組（17.6%），F183 組（33.6%），D179 組（32.8%）であった．F と D の共通救済は 139 組，F のみ 44 組，D のみ 40 組であり，救済数が近くても対象は異なった．F のみ・D のみの救済計 84 組について，両条件 168 履歴・404 修正遷移を静的に確認した．
独立再生成 B4 では同じ初期コードを保持し，初期失敗に対して最大 3 回，前回コードや診断を与えず再生成した．B4 は 216/545 組（39.6%）で成功し，最終成功は 671/1,000 組（67.1%）であった．追加 1,343 要求，996,223 tokens，追加 tokens／追加成功は 4,612.1 であった．最大生成回数が同じでも入力情報・seed 規則・実行時期が異なり，因果効果は断定しない．
未救済例への消費を含む追加 tokens／救済は R18,852.6，F9,333.8，D11,037.9 であった．3 回目の修正は F で 28 組，D で 32 組を追加救済した．これらは保存記録と部分追試を組み合わせた計算値であり，全件新実験の実測コストではない．結果は今回のモデル・タスクと指定入力での評価に限られ，介入の一般的優劣，因果効果，統計的有意差を主張しない．今後は介入選択・停止方策を事前固定し，別 seed・タスクで検証する必要がある．

目次

1 はじめに 5
1.1 研究背景 5
1.2 研究目的と研究課題 5
1.3 論文の構成と対象範囲 5
1.4 コード生成支援から失敗後の意思決定へ 5
1.5 初期失敗を基準とする研究の意義 6
1.6 用語と主張の範囲 6
2 関連研究 8
2.1 量子プログラミングと Qiskit 8
2.2 自己フィードバックによる修正 8
2.3 実行結果と自己デバッグ 8
2.4 量子回路コードで確認する対象 9
2.5 自己修正を評価する際の外部情報の役割 9
2.6 量子コード生成ベンチマークとの相違 9
2.7 反復生成と再生成対照の位置づけ 10
3 評価・実験方法 11
3.1 介入条件と評価指標 11
3.1.1 6 条件と共有初期コード 11
3.1.2 独立再生成対照 B4 12
3.1.3 L3 と出力一致 12
3.1.4 救済・保持・独立生成の比較 12
3.1.5 量子状態・測定分布とコードの正しさ 12
3.1.6 段階評価を用いる理由と診断の情報量 13
3.1.7 出力距離・測定順序・構造要件の詳細 13
3.1.8 測定禁止修正と厳格評価の位置づけ 14
3.1.9 対応集合による指標の定義 14
3.2 実装と記録の検証 15
3.2.1 ハーネスの構成 15
3.2.2 失敗分類とモデル接続障害 15
3.2.3 到達履歴の再構成 15
3.2.4 集計の整合性と静的履歴分析 15
3.2.5 生成・評価・停止を結ぶ処理の流れ 16
3.2.6 履歴再構成における分岐と到達性 16
3.2.7 記録の検証と失敗の切り分け 16
3.2.8 静的履歴分析の観察単位 17
3.3 実験設定と出典 17
3.3.1 対象モデルと課題 17
3.3.2 生成条件と元実験の区別 18
3.3.3 追加コストと修正回数の集計方針 18
3.3.4 課題集合の役割と被覆範囲 18
3.3.5 生成設定を揃える意味と限界 19
3.3.6 分析対象と固定出典の範囲 19
4 結果 20
4.1 全体の結果と比較の前提 20
4.2 モデル別の結果 21
4.3 初期失敗類型別の結果 22
4.4 タスク・難度別の結果 22
4.5 成功対象の重なりと履歴分析 23
4.6 追加計算コストと停止回数 24
4.7 P の独立生成としての比較 25
4.8 補足実験：失敗通知と詳細診断の比較 25
5 考察 26
5.1 修正と独立再生成の比較から得られる知見 26
5.2 介入カタログの構成と適用範囲 26
5.3 資源制約と停止方策への含意 27
5.4 妥当性と限界 28
5.5 評価漏れの修正範囲と残る制約 28

5.6 今後の検証計画 ... 28
6 おわりに ... 30
6.1 研究成果 ... 30
6.2 今後の課題 ... 30
7 参考文献 ... 31
8 付録 ... 32
8.1 出典と再現性 ... 32
8.1.1 5 モデル追加実験の保存設定 ... 32
8.1.2 3 モデル本実験の保存設定 ... 32
8.1.3 2 モデル補完実験の保存設定 ... 32
8.1.4 固定集計と公開範囲 ... 32
8.2 指標の読み方 ... 33
8.2.1 集計単位 ... 33
8.2.2 退行 0 の意味 ... 33
8.2.3 静的変更の意味 ... 33
8.2.4 コストの意味 ... 33
8.3 実験プロンプト ... 34
8.3.1 共通 system メッセージ ... 34
8.3.2 課題仕様 T1_Bell ... 34
8.3.3 課題仕様 T2_GHZ ... 34
8.3.4 課題仕様 T3a_DJ_constant ... 34
8.3.5 課題仕様 T3b_DJ_balanced ... 35
8.3.6 課題仕様 T4_BV_011 ... 35
8.3.7 課題仕様 T5_Grover_11 ... 35
8.3.8 課題仕様 T6_QFT_001 ... 36
8.3.9 課題仕様 T7_IQFT_001 ... 36
8.3.10 課題仕様 T8_QPE_001 ... 36
8.3.11 課題仕様 T9_Ansatz_001 ... 37
8.3.12 修正テンプレート R：自己見直し ... 37
8.3.13 修正テンプレート F：実行結果提示 ... 37
8.3.14 修正テンプレート D：自己デバッグ ... 38
8.3.15 P の予防的注意事項 ... 38

1 はじめに

1.1 研究背景

量子回路をプログラムとして構築する際には，量子状態や測定分布に関する理解と，ソフトウェア API の正しい使用の両方が必要になる．LLM の生成コードを利用する場合も，構文上の正しさだけで有用性を判断できない．import が成功しても回路構築に失敗する場合があり，実行が完了しても期待した出力と一致しない場合がある．
生成を一度で終えず，前回コードを見直させたり，実行結果を提示したりする方法が考えられる．自己フィードバックによる反復改善の Self-Refine [1]と，コードの説明や実行結果を用いた Self-Debugging [2]は，こうした修正の背景となる．ただし，一般的なコード生成での知見が，量子回路コードの各種失敗に同じ形で当てはまるとは限らない．
実行結果には，存在しない API の使用を知らせる診断と，期待出力との不一致を知らせる診断が含まれる．両者では修正に必要な情報が異なる可能性がある．全体の最終成功率だけを比較すると，初期成功をそのまま保持した分と，修正によって救済した分が混在する．そこで本研究では，初期失敗を分母に固定して介入を比較する．
モデル・タスク・seed の組合せを初期組と呼ぶ．本研究では，最終評価水準 L3 の要件をすべて満たすことを「L3 成功」と呼ぶ．現行評価器の停止・修正規則に従って到達可能な生成履歴を，到達履歴と呼ぶ．

1.2 研究目的と研究課題

本研究の目的は，Qiskit 量子回路コードの初期失敗に対する修正介入と独立再生成の有効性を比較し，失敗類型やモデルに応じた対処方法を選択するための根拠を整理することである．
RQ1（追加成功）　修正介入 R/F/D と独立再生成 B4 では，同じ初期失敗のうち，どれだけの事例が追加の生成によって L3 に到達するか．
RQ2（救済対象）　救済数が近い介入は，同じ初期失敗事例を救済するか．
RQ3（追加計算コストと修正回数）　未救済例も含む追加計算コストと修正回数は，救済数とどのように対応するか．
本研究の貢献は，共有初期コードを用いた修正介入と独立再生成の追加成功率の比較，初期失敗類型・モデル・タスク別の初期失敗数を明示した分析，F と D の片側救済全 84 組の履歴整理，および追加計算コストと 3 回目の寄与の記述である．介入の最適な選択方策を既に実証したとは主張しない．

1.3 論文の構成と対象範囲

第 2 章で関連研究を整理し，第 3 章で評価・実験方法として介入条件・評価指標・実装・実験設定を述べる．第 4 章で結果，第 5 章で考察，第 6 章で結論と今後の課題を述べる．第 7 章に参考文献，第 8 章に出典，指標の読み方，実験プロンプトを付録として示す．B/R/F/D/P の元実験，必要な部分追試，B4 の追加実験は実施時期が異なる．本稿の主結果は，これらの記録に回路内測定禁止の検査と必要な後続生成を反映した，2026 年 11 月 4 日の測定禁止修正版の固定集計 [3] に基づく．各実験の由来と保存設定は第 3.3 節と付録 8.1 節に示す．

1.4 コード生成支援から失敗後の意思決定へ

自然言語からプログラムを生成できることと，そのプログラムを目的に沿って利用できることは同じではない．利用者は生成されたコードを読み，実行環境に接続し，結果を確認し，問題があれば次の操作を選ばなければならない．コード生成支援の実際の作業では，最初の出力の品質だけでなく，失敗を確認した後に何を入力し，どこで生成を止めるかが作業量を左右する．本研究はこの失敗後の意思決定に対象を絞る．
コード生成の機能的正しさを評価する HumanEval [4]や，自然言語から短い Python プログラムを生成する MBPP [5]は，生成したコードを実行し，機能を確認する評価の背景となる．ただし，本研

究はこれらの課題をそのまま利用したベンチマークではない．対象を独自に定義した量子回路課題に限定し，初期失敗の後に行う修正と再生成を対応づけて調べる．課題集合や評価基準が異なるため，他のベンチマークにおける成功率との直接比較は行わない．
例えば，存在しない関数名を import して失敗した場合には，実行診断から変更箇所の候補を狭められる可能性がある．一方，コードは実行できるが量子状態が異なる場合には，単に例外が出ないように書き換えるだけでは不十分である．必要な情報や修正箇所が異なる以上，すべての失敗へ一つの指示を一律に適用することが適切かは実証的に確認すべき問題になる．この二つの例は介入の役割を説明するための想定例であり，特定の保存事例を新たに再現した結果ではない．
失敗後の操作には，既存コードを残して直す方法と，既存コードを捨てて生成し直す方法がある．修正には，前回の出力を再利用できる利点がある一方，誤った構造を引き継ぐ可能性もある．再生成には，既存の誤りから離れられる可能性がある一方，既に正しかった部分を再び作り直すことになる．これらは設計上考えられる性質であり，本研究が個別の心理過程やモデル内部の挙動を直接測定したという意味ではない．実際の比較では成功数と追加計算コストを観測して判断する．
本研究では，コード生成時または生成後の再試行時に，LLM へ与える情報や指示を変更する操作を介入と呼ぶ．生成前に注意事項を与える P と，生成後に修正を促す R/F/D を含む．一方，B4 は前回コードや診断，修正指示を与えず，同じ課題仕様から独立に再生成する対照条件である．モデルの重みを更新する学習ではなく，同じ課題を解く過程での入力の構成に着目する．したがって，結果の解釈にはモデル識別子だけでなく，プロンプト，評価器，停止条件，生成機会上限を含める必要がある．介入名だけを取り出して一般的な効力を論じることは避ける．

1.5 初期失敗を基準とする研究の意義

最終成功率のみを用いると，初期から正しかったコードの割合と，初期に誤っていたコードを修正できた割合が混在する．初期性能が高いモデルは修正対象が少なく，初期性能が低いモデルは修正対象が多い．この違いを考慮せずに最終成功数を比べると，介入が働いた範囲を読み取りにくい．本研究は各組の初期判定を保存し，その初期失敗集合に対して修正後の成功を数えることで，修正が必要だった対象に対する成績を明示する．
初期コードの共有は，比較の出発点を揃えるためにも必要である．別々の初期生成から R と F を始めた場合，R 側の初期失敗が軽く，F 側の初期失敗が重い可能性を排除できない．本研究ではモデル・タスク・seed で識別される初期組を基準に，R/F/D の最初のコードと判定を一致させる．この対応づけは比較の解釈を助けるが，後続生成の乱数や入力長，実行時期まで等しくするものではない．共有初期は比較の必要な条件の一つであり，単独で因果推論を成立させる条件ではない．
また，同じ数の失敗を救済したとしても，その集合が一致するとは限らない．二つの条件がそれぞれ多くを救済しても，片方だけが救済できる対象があれば，実行前にどちらを選ぶかという問題が残る．反対に，成功対象がほぼ重なるのであれば，追加計算コストや実装の簡単さを重視できる可能性がある．本研究では数と集合の両面を扱い，単一の順位表では捉えにくい介入間の関係を記述する．
本研究の実証的貢献は，選択方策の完成ではなく，選択方策を設計する前に必要な観測単位を整えた点にある．初期失敗，修正履歴，最終成功，追加計算コストを初期組へ結び付けることで，後から条件の選び方を検討するための根拠を残す．この段階の知見を，そのまま実運用における最適な規則へ置き換えない．記述的な結果を整理する段階と，方策を新しい対象で検証する段階を区別すること自体が，結果を過大解釈しないための研究設計である．

1.6 用語と主張の範囲

本稿の「成功」は L3 を満たすという操作的な意味で使用する．利用者のあらゆる意図を満たすことや，任意の量子入力に対して同じ算法を実装することを表す用語ではない．「失敗」はこの評価基準への未達を表し，モデルの一般的能力の欠如を意味しない．評価基準と用語を対応づけることにより，どの判断を機械的に検査し，どの判断を今後の検討に残しているかを区別する．
「救済」は，同じ初期失敗コードを R/F/D で修正し，最終的に L3 へ到達した場合に用いる．B4 の成功は，初期組を共有するものの失敗コード自体を修正しないため，「独立再生成による追加成

功」とする．P では初期コードを共有しない独立 1 回生成の成績を，B との対応で記述する．これらは最終成功という同じ判定を利用していても，出力を得る手続きが異なる．
「コスト」は，ここでは主にモデル要求に記録された tokens と生成要求時間を指す．利用者の読解時間，電力，実行装置の償却，金銭単価は測定していない．追加 tokens が少ないという観測を，人間の総作業量が同じ割合だけ減ったという主張に置き換えない．同様に，「介入効果」という表現を用いる箇所でも，原則として対応づけた観測上の変化を指し，無作為化比較によって特定された因果効果とは区別する .

2 関連研究

2.1 量子プログラミングと Qiskit

Qiskit は，Python を用いて量子回路を記述するためのソフトウェア開発キット（SDK）である．量子回路は，量子ビットと，それらに適用するゲートや測定などの命令から構成される．Qiskit では QuantumCircuit クラスを用いて回路を表し，量子ビット数などを指定して作成した回路へ命令を追加する．例えば，h メソッドはアダマールゲートを，cx メソッドは制御 X ゲートを追加する [8]．生成するコードには，Python としての正しい記述に加え，課題に応じたゲートの選択と適用対象の指定が必要となる．
具体例として，初期状態|00⟩から，2 量子ビットの Bell 状態(|00⟩ + |11⟩)/√2 を準備するコードを示す．量子ビット 0 へアダマールゲートを適用し，続いて量子ビット 0 を制御，量子ビット 1 を標的とする制御 X ゲートを適用する．両量子ビットを計算基底で測定すると，理想的には測定結果のビット列「00」と「11」がそれぞれ確率 0.5 で得られる．以下は T1 の課題仕様に基づく説明用コードであり，モデルの生成結果の抜粋ではない．
コード例（Bell 状態の回路構築）

```
from qiskit import QuantumCircuit

def build_circuit() -> QuantumCircuit:
    qc = QuantumCircuit(2)
    qc.h(0)
    qc.cx(0, 1)
    return qc
```

この関数は回路を返すだけで，測定やシミュレータの実行を含まない．本研究では測定を必要に応じて評価器側で追加し，課題仕様への適合を判定する．
回路を記述することと，その回路を実行して結果を得ることは別の処理である．Qiskit Aer は量子回路のシミュレーションを行うためのソフトウェアであり，実機を用いずに回路の動作を調べる手段を提供する [9]．本研究では，LLM が生成したコードから回路を構築し，タスク仕様に応じて測定分布または状態ベクトルを期待出力と比較する．生成コードには build_circuit()を定義して QuantumCircuit を返すことを求め，評価処理はハーネスが担当する．したがって，本研究の成功判定は実機上での動作を検証した結果ではない．
この構成では，必要なモジュールを読み込めない import 失敗，API の使い方などにより回路を作れない回路構築失敗，回路を評価できても期待状態や分布を得られない出力不一致を区別できる．同じ量子コード生成の失敗でも，修正時に利用できる診断と，確認すべき内容は異なる．これらを区別したうえで，以下では自己フィードバック，実行診断，量子コード生成の評価に関する先行研究を整理する．

2.2 自己フィードバックによる修正

Self-Refine [1] は，同じ LLM による生成，フィードバック，修正の反復を扱う．本研究の R は，前回コードと課題仕様を与えて自己点検を求める条件であり，外部評価の診断内容を与えない．自己点検だけで救済できる範囲を確認する比較条件として用いる．
原論文と本実装ではプロンプト構成，対象課題，出力制約，モデルが異なる．したがって本研究の R の結果を，原論文の手法全体の性能と同一視しない．先行研究は設計の背景であり，完全再現実験として扱わない．

2.3 実行結果と自己デバッグ

Self-Debugging [2] は，生成コードの説明や実行結果の検討を用いた自己デバッグを扱う．本研究では F を実行ベースの診断を用いる条件，D を同じ評価方式に基づく診断に加えて説明・原因分析・修正の順序を促す条件として実装した．D の追加指示と出力長の違いを含めて観測する．

D は説明と原因分析を修正版コード先頭のコメントに残す実装である．モデルが内部でどのように推論したかは測定していない．D で救済されたという事実だけから，説明が正しかった，あるいは説明そのものが成功原因だったとは判断できない．

2.4 量子回路コードで確認する対象

本研究は，回路を返す関数の存在，回路の構築と評価処理，指定入力の期待状態または分布，所定の構造要件を区別する．量子回路では，実行可能性と期待出力への適合は別の性質である．
本研究は，選定した量子回路課題における介入の救済対象と追加計算コストを記述する実証研究である．一般の量子プログラム全体への網羅的評価や，形式的な同値性証明は対象に含めない．その範囲を明確にしたうえで，成功例と失敗例を同じ分母で扱う．

2.5 自己修正を評価する際の外部情報の役割

自己修正という呼び方には，モデル自身の出力だけを見直す場合と，実行や採点から得た外部情報を使う場合が含まれ得る．本研究では両者を混ぜないため，R と F/D を分けた．R は課題仕様と前回コードに基づく自己点検であり，F/D では検証器が出力した情報が次の要求に入る．同じモデルを使っていても，入力として利用できる情報が変わるので，これらを同じ意味での自己修正と扱うことは適切でない．
Huang ら [6] は推論課題において，外部フィードバックのない自己修正の困難さを論じている．本研究では，自己点検により常に改善すると仮定しないための背景として参照する．同論文が本研究の量子コードや選定モデルで同じ結果を示したと主張するものではない．対象課題，モデル，評価指標の違いを踏まえ，本研究では外部情報の有無を比較条件として明示する．
外部情報がある場合にも，提示した情報が正しく対象の失敗を指していることが前提となる．評価器のビット順処理や判定条件が誤っていれば，正しい回路に不適切な修正を促す可能性がある．したがって，診断を利用する手法の評価では，診断の有無だけでなく，その診断を作った評価器の仕様と検証履歴を記録しなければならない．本研究が評価方針の修正と到達履歴の再構成を方法へ含めるのは，評価器を単なる受動的な採点装置と扱えないためである．
F と D の相違は説明・原因分析の指示にあるが，この差は入力文の長さや出力コメントにも関係する．そのため，最終成功数の差を説明行の存在だけへ帰属できない．説明が誤っていても偶然にコードが改善する場合や，説明が妥当でも実装が失敗する場合は概念的に区別できる．本研究が直接観測するのは要求と出力と評価の系列であり，内部推論の内容や説明の忠実性を測定する実験ではない．

2.6 量子コード生成ベンチマークとの相違

Qiskit HumanEval [7] は，Qiskit コード生成を評価するための課題群を提示している．量子 SDK を対象としたコード生成でも，生成物の実行と正しさを検証する専用の課題設計が必要になることを示す関連研究として位置づける．本研究の 10 タスクは，同ベンチマークから抽出したものではなく，独自の課題仕様，期待出力，構造要件に基づく．名称が近い評価であっても，課題の出典と採点の定義を分離して記述する．
本研究では広範な量子 API の被覆率を最大化するのではなく，同じ初期失敗に複数の介入を適用し，その救済対象を対応づけることを優先した．そのため，課題数を増やした一般的なモデルランキングとは目的が異なる．課題の種類が少ないことは一般化の制約である一方，個々のタスク仕様や履歴を追いやすいという分析上の特徴がある．
量子 SDK のコードには，Python の文法と API 利用に関する正しさに加え，構築した回路が目的の状態や分布を生成するという正しさがある．SDK 上での実行可能性だけを検査した場合と，指定出力まで検査した場合では，成功集合が異なり得る．したがって，本研究の L3 成功率を他研究の実行成功率と同じものとして比較しない．比較に必要なのは，判定対象，入力範囲，測定方式，失敗時の扱いを明らかにすることである．

2.7 反復生成と再生成対照の位置づけ

同じ課題から複数候補を生成する方法は，単一候補では得られなかった成功を得るための比較対象になる．ただし，複数候補の中から成功したものを選ぶには，候補を評価する手続きも必要である．本研究の B4 は，各候補を L3 で検査し，成功時に停止する対照である．候補の生成だけでなく，評価と停止も含む手順として比較する．

本研究の B4 は，既存の初期生成を共通に保持し，失敗した組だけ追加生成する．このため，課題ごとに初めから 4 個の独立候補を一括生成した一般的な設定と完全には一致しない．初期コードの出典，追加生成用の seed 規則，成功時停止という手続きが成績へ関係する．本稿では B4 という操作条件の名前で報告し，別の推定量や他研究の指標と無条件に同一視しない．

修正条件が単回生成より高い成績を得ても，それだけでは診断や前回コードの再利用が役立ったことの証拠にはならない．単に生成要求を追加したことによる機会の増加が含まれるためである．そこで，本研究では初回込み最大 4 回という上限を B4 と R/F/D で対応づける．ただし，生成回数の上限を揃えても，入力情報量や消費 tokens は一致しない．

3 評価・実験方法

図 1 に実験の論理的な比較手順を示す．初期成功の保持と初期失敗への追加生成を分け，生成前介入 P は点線の独立経路で示す．保存済みの生成履歴と部分追試に基づく集計であり，全件を新たに生成した実験ではない．

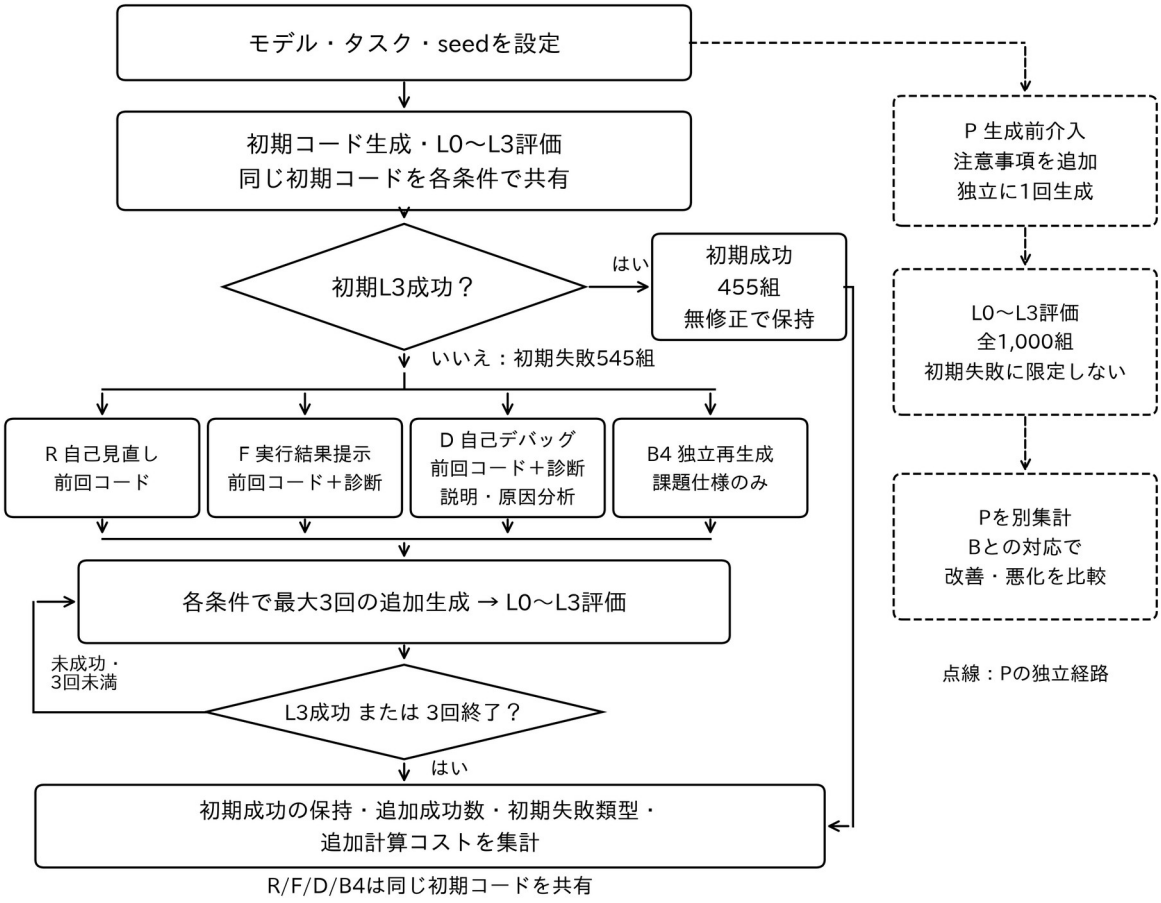


図1 共有初期コードを起点とする比較手順

3.1 介入条件と評価指標

3.1.1 6 条件と共有初期コード

表 1 に 6 条件の方法，最大生成回数と役割を示す．モデル・タスク・seed の組を初期組と呼ぶ．B は初期生成のみの条件である．R/F/D は B と同じ初期コードと判定を共有し，初期失敗に対して最大 3 回の修正を行う．成功した時点で停止し，以降の修正は行わない．R/F/D/B4 の最大 4 回は，初回 1 回と最大 3 回の追加生成を意味する．B4 も B の初期コードを共有する一方，P は B とは異なる初期コードを独立に生成する．

R は自己見直し，F は実行結果提示，D は説明・原因分析を促す自己デバッグである．P は初回プロンプトに注意事項を追加して独立に生成する条件であり，同じコードを修正する条件ではない．P の対応比較は，B の初期失敗に対応して成功した場合と，B の初期成功に対応して失敗した場合を分ける．

表1 6 条件の方法・最大生成回数・役割

条件	方法	最大生成回数	役割
B	初回生成のみ	1	基準条件
P	生成前注意事項付き生成	1	生成前介入
R	自己見直し	4	修正介入
F	実行結果提示	4	修正介入
D	自己デバッグ	4	修正介入
B4	独立再生成	4	生成機会上限をそろえた対照条件

3.1.2 独立再生成対照 B4

B4 は初期 1 回を含む最大 4 回生成である．B と同じ初期コード・L3 判定を用い，初期成功 455 組は保持する．初期失敗 545 組のみ，前回コードと診断を与えず課題仕様から最大 3 回独立再生成し，L3 成功時に停止する．初期失敗からの成功は修正介入による救済と区別する．
5 モデル・10 タスク・base seed 0〜19 は従来と対応するが，追加生成は independent_4_v1 の専用 seed 規則を用いる．temperature 0.7，max_tokens 1,024，timeout 600 秒，並列 1，reasoning_effort none で実行した．上限が同じでも入力情報・実要求数・tokens・実行時期・診断文・出典 schema は等しくない．

3.1.3 L3 と出力一致

L0 は生成コードの読込み・import 等と必須インターフェースの要件，L1 は回路構築と評価処理の実行，L2 は指定入力に対する期待出力との一致を表す．L3 は L0，L1，L2 および structure_match がすべて真であることと定義する．構造要件には量子ビット数と測定・barrier を除く回路命令数の範囲を用いる．
測定分布の距離は全変動距離 $TVD(p,q) = (1/2)\Sigma x\ |p(x)-q(x)|$ とし，8,192 shots で得た分布と期待分布を比較する．状態ベクトルは大域位相を整合したユークリッド距離を用いる．各タスクで距離 0.05 未満を出力一致とする．測定・変換 seed は 42 に固定したが，有限 shots と指定入力に限られる．
本稿で用いる evaluation_policy_v4 は，測定先から量子ビット順への対応を反映する．タスクに構造制約が明示されていない項目を，一律の制約として追加しない．L3 は所定の検査を通過したことを意味し，全入力でのアルゴリズム同値性やプロンプトの全自然言語条件の充足を保証しない．

3.1.4 救済・保持・独立生成の比較

R/F/D による修正で，初期 L3 未達から最終 L3 成功に変化した初期組を救済と呼ぶ．救済率 = 救済組数／初期 L3 未達組数 とする．初期失敗類型は介入前の B の分類に固定する．修正後の分類で分母を選別すると対象が変わるため，その方法は採用しない．
R/F/D の最終成功には初期成功の無修正保持が含まれる．成功時停止により退行 0 となる設計であり，退行 0 をモデルが常に正しさを維持できる証拠と解釈しない．P では独立生成による改善と悪化の両方を報告する．

3.1.5 量子状態・測定分布とコードの正しさ

本研究の出力評価を理解するには，回路が表す処理と，回路を実行して得られる観測結果を区別する必要がある．本研究で状態ベクトル評価の対象とする純粋状態は，計算基底に対する複素振幅を成分とする正規化された状態ベクトルで表され，計算基底での各測定結果の確率は対応する振幅の絶対値の二乗となる．したがって，同じ測定確率を持つ純粋状態であっても，相対位相が異なる可能性がある．このことから，分布を検査するタスクと状態ベクトルを検査するタスクでは，確認できる正しさの範囲が異なる．
説明用の例として，$|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$ と $|-\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$ を考える．両状態は計算基底での測定分布が同じでも，相対位相が異なる．それぞれにアダマールゲートを適用してから計算基底で測定

すると，前者ではビット値 0，後者ではビット値 1 が確率 1 で得られる．この例は評価原理を説明するためのもので，保存コードを新たに実行して得た観測ではない．分布の一致だけから位相まで含めた状態の同一性を結論しないことが，本研究の評価範囲を正しく読むために必要となる．
一方，状態ベクトルの比較では，すべての振幅へ同じ位相を掛ける大域位相を区別しないようにする．本評価器では，実際の状態と期待状態の内積から位相差を求め，その差を整合した後の距離を計算する．単純な複素ベクトルの成分比較では，物理的に区別しない大域位相によって距離が大きくなる場合があるためである．ただし，相対位相の違いを大域位相の整合で打ち消すことはできない．
さらに，指定入力で出力状態が一致していても，回路全体が任意入力で同じ変換を行うとは限らない．ある一つの入力に対して同じ出力を返す二つの回路が，別の入力では異なる出力を返すことはあり得る．本研究の状態評価はタスクに指定された入力に対する評価であり，ユニタリ行列全体の同値性を証明する検査ではない．この区別は，QFT や IQFT など算法名を持つ課題を評価する際にとくに重要である．
構造条件は，出力検査だけでは捉えない課題上の制約を補うために用いる．例えば量子ビット数が仕様と異なる場合には，指定された実装要件を満たしていない．ただし，回路の命令数が範囲に収まることを検査しても，算法の意味を十分に特徴づけたことにはならない．構造検査は指定した必要条件の確認であり，出力検査と合わせて L3 を構成する．構造要件へ合格したことを形式検証の完了と表現しない．

3.1.6 段階評価を用いる理由と診断の情報量

段階評価では，どこで検査が止まったかにより，確認済みの性質が異なる．構文や import で止まったコードに対しては，回路の量子出力が正しいかどうかを観測できない．逆に，出力不一致まで進んだコードでは，少なくともコード読込みと回路の構築・評価処理が通過している．すべてを失敗の一語へまとめると，介入が利用できる手掛かりや，修正の到達点を区別しにくくなる．
L0 は構文・import などの読込みと，必須関数を利用できるインターフェースを確認する．関数名が書かれているだけでなく，呼び出し可能であることや，要求された呼出方法に適合することが必要になる．この段階では，回路を実際に構築できることまでは確定しない．L0 を通過したことを，量子コードとして正しいことと同一視しない．
L1 は回路構築と評価処理を実行できる段階である．build_circuit()が回路を返すことや，未束縛パラメータなど評価を妨げる条件がないことも，実装上の確認に関係する．ただし，評価処理を実行できることと，その結果が期待どおりであることは別である．L1 成功から L2 失敗へ進んだ場合には，単に例外を取り除く修正では不十分だった可能性がある．本稿では可能性の説明と個別の原因認定を区別する．
L2 は指定入力に対する期待出力と実測または計算された出力の距離を比較する．L3 はこの出力適合性に，所定の構造条件を加える．一つの最終判定へまとめる場合でも，各段階の情報を保持することで，実行不能から実行可能へ変わったが出力はまだ誤っているという途中の改善を記録できる．ただし，本研究の主要指標である救済では最終 L3 成功を要求するため，途中段階の改善を成功数に加算しない．
診断の情報量は失敗段階によって異なる．構文の位置や import 名は変更候補を直接示す場合がある一方，期待分布との不一致は多くのゲート操作や測定の取り方に関係し得る．この違いから診断が常に同じ難しさで利用できるとは仮定しない．また，失敗分類は最初に露見した問題を表す場合があり，コード内のすべての欠陥を一つずつ列挙したものではない．初期分類を因果的な根本原因の完全な記述として扱わない．

3.1.7 出力距離・測定順序・構造要件の詳細

測定分布の比較では，期待分布と観測分布のキーの和集合を取り，各ビット列の確率差の絶対値を合計して半分にする．出現しなかったビット列の観測確率は 0 として扱う．この定義により，期待していなかった出力へ確率が流れた場合も距離へ反映される．本評価では距離が 0.05 未満であることを出力一致とし，ちょうど境界値となる場合は成功へ含めない．

8,192 shots の観測分布には標本変動が含まれる．同じ理想回路でも有限回の測定から得た頻度は期待確率と完全には一致しないため，閾値を設けて比較する．測定 seed の固定は再実行時の変動を抑えるための設定であり，別の seed でも同じ合否になることを証明するものではない．本研究では複数の測定 seed に対する合否の安定性を新たに測定しておらず，閾値近傍の感度分析は今後の課題とする．
測定結果の文字列表現を比較する際には，量子ビットと古典ビットの対応を確認する必要がある．コードがどの量子ビットをどの古典ビットへ測定したかに応じて，結果キーを課題の量子ビット順へ戻さなければ，単なる表現上の並びの違いを量子出力の失敗と誤判定する可能性がある．evaluation_policy_v4 はこの対応を反映する評価方針であり，文字列を無条件に反転すればよいという規則ではない．
状態ベクトル評価では，指定入力から得る複素振幅全体を，大域位相を整合して比較する．既存の評価器は，入力状態の準備を含む回路を Qiskit の Operator へ変換し，ユニタリ性を確認したうえで，全ゼロ状態からの出力を計算する．したがって，reset や initialize などの非ユニタリ操作を含む回路の一般的な状態評価は対象外である．これは任意入力に対するユニタリ全体の同値性の検査ではない．
T6・T7・T9 の期待状態と T8 の期待分布は，課題別の参照コードが構築する回路を Qiskit の Statevector で計算したものである．期待値の生成と候補の評価に同じ SDK を用いるため，得られた一致は参照回路と SDK に依存する．独立した数値実装による数学的検証とは区別する．
構造要件のうち回路命令数は，測定と barrier を除いて数える．評価用の読出し操作を追加したことだけで算法本体の命令数が増えたと判定しないためである．量子ビット数や命令数の範囲など元の構造要件に加え，共通 system で禁止された build_circuit()が返す回路内の測定を明示的に検査する．旧 evaluation_policy_v4 は個別タスクの測定メタデータが未指定の場合に共通禁止を反映せず，命令数から測定を除く処理も違反の検出にはならなかった．修正版 evaluation_policy_v4_measurement_ban_v1 は，評価用読出しを追加する前の回路について，実際の Measure 命令を命令定義・制御フローブロックまで再帰的に確認し，違反時は構造要件不適合として L3 を不合格にする．

3.1.8 測定禁止修正と厳格評価の位置づけ

今回の測定禁止検査は原課題の共通 system に既にある条件の評価漏れを補うものである．位相まで含む分布課題の状態一致，全ユニタリ同値性，古典ビットや reset の一律禁止など，原課題より強い参照適合性検査は主 L3 へ追加していない．別実装の厳格評価は補助的な検証候補として区別し，本稿の成功数には用いない．
旧 L3 成功コード 1,561 組（コード hash とタスクの組）を同じ回路の旧評価と修正版評価で確認した．旧評価の成功を全件再現し，測定違反 4 組を検出した．旧評価で既に失敗した試行は，今回の条件追加で成功には変わらない．後続要求を不要に変更しないため，既存の失敗診断の優先順位を保持した．旧成功が失敗へ変わる場合だけ，測定禁止の診断を採用する．

3.1.9 対応集合による指標の定義

初期組の集合を G とし，そのうち B の L3 判定が失敗だった部分集合を U とする．本研究では G の大きさが 1,000，U の大きさが 545 である．条件 c により U の中から最終成功へ到達した集合を S(c)と書けば，修正条件の救済率は S(c)の要素数を 545 で割った値となる．この表記は，条件間で同じ初期対象を比較することと，最終成功集合の全体を混同しないために導入する．
R/F/D では初期成功集合を無修正で保持するため，最終成功数は 455 と救済数の和である．例えば F の最終成功 638 組は，初期成功 455 組と救済 183 組を合わせた数になる．638 を 545 で割ると初期成功を分子へ含めた不適切な率になり，183 を 1,000 で割ると初期失敗に対する救済率ではなく全初期組に対する追加成功割合になる．同じ件数から複数の割合を計算できるため，指標名だけでなく分母を明示する．
F と D の比較では，S(F)と S(D)の共通部分，F 側だけ，D 側だけ，どちらにも含まれない部分へ U を分割する．この四分割は互いに排他的で，合計は 545 になる．両条件が同じ初期組に適用され

ているため，周辺の成功率だけでなく，個々の対象が一致しているかを調べられる．全体差が小さくても片側の成功が多数あるという結果を，この四分割が捉える．
P では B のコードを修正しないので，B で失敗だった対象に対応する P の成功と，B で成功だった対象に対応する P の失敗を別に数える．B4 では初期成功を保持し，初期失敗への追加候補を評価するため，失敗対象からの追加成功集合を S(B4)として対応づけられる．ただし，集合の記法が同じでも操作内容は同じではない．記号上の比較の便利さを理由に，修正と独立生成の違いを消さない．
層別分析では，モデル，タスク，初期失敗類型などで U を分割し，各層の分母と成功数を示す．全体率は層別成功数の合計を層別分母の合計で割った値であり，各層の率の単純平均とは一般に一致しない．少数の失敗しかない層で高い率が得られても，全体に同じ比率で寄与するとは限らない．本研究のカタログでは，率の大小だけで判断せず，対象数も一緒に読める形を基本とする．

3.2 実装と記録の検証

3.2.1 ハーネスの構成

実装は Python を中心とし，tasks/が課題仕様・期待値・構造条件を，harness/interventions/prompts.py が条件別の修正指示を保持する．harness/llm_clients.py がモデル要求を扱い，harness/experiment.py 等が初期共有と反復を管理する．harness/evaluator.py と harness/readout.py が回路と出力を検証する．これらの責務を分け，生成の失敗と評価の不具合を区別して追跡する．
評価対象コードは build_circuit()を定義し，QuantumCircuit を返すインターフェースを前提とする．測定分布と状態ベクトルのどちらを検査するかはタスク仕様で定める．実装環境の依存範囲は requirements.txt に保持し，過去の実験の実行環境を現在の環境から推定して埋めない．保存設定・ハーネス版・モデル digest は元実験 manifest を参照する．

3.2.2 失敗分類とモデル接続障害

構文，import，必須インターフェース，回路構築，出力不一致，量子ビット数などの分類を記録する．コード側の import_error や build_error は介入の対象となる実験結果である．一方，モデル API の接続エラーやタイムアウトは生成されたコードの品質と分けて検証する．
保存時に unknown_error と記録された初期失敗 1 組について，事後確認により未定義変数 sim による NameError と判明したため，本稿では「実行時エラー（未定義変数）」と表記する．保存データの分類キーは変更していない．API 接続障害には含めない．固定出典の検証状態を参照し，未解決の実行・評価問題を確定結果のように扱わない．

3.2.3 到達履歴の再構成

評価方針の修正は，停止する時点や次に与える診断文を変えることがある．保存コードを新しい評価で再採点するだけでは，新診断を受けて生成されるはずの後続コードを再現できない．そこで，検証済みの生成履歴の前半部分と，必要な部分追試による後半部分を接続した現行の到達履歴を主結果とする．
部分追試は 293 ケースで，部分追試で生成した後半履歴は 647 ラウンドである．分岐後に到達できない旧保存後半部分を現在の経路に流用しない．初期生成から全件を新評価で生成し直した実験とは区別する．現行の到達 round は 9,249 であり，5,000 条件記録とは集計単位が異なる．

3.2.4 集計の整合性と静的履歴分析

固定集計では全 5,000 条件の対応，欠落・重複・予定外記録，共有初期の一致，成功時停止，層別分母の合計，出典と到達コードの hash を検査した．固定 sanity report は成功しており，欠落・重複・予定外記録・共有初期不一致は 0 である．主分析の表は，2026 年 11 月 4 日の測定禁止修正版の固定集計の公開 CSV から作成した．
片側救済 84 組については両条件 168 履歴，404 修正遷移をコード文字列として静的確認した．import/API，回路構築・インターフェース，量子ゲート等のテキスト，測定・出力取得を複数

該当可で注記した．生成コードを実行しない分析であり，変更の意味や因果的寄与は別の検証を要する．

3.2.5 生成・評価・停止を結ぶ処理の流れ

処理は，課題仕様とモデル設定の読込み，初期コードの生成，コードの抽出と評価，条件別の次要求の作成，停止判定，記録の保存という流れで構成する．生成要求の回数と，保存された条件記録数は異なる．一つの条件記録は初期コードと複数の修正 round を含み得る．条件の最終結果だけを取り出しても，そこへ到達するためにどれだけの生成を行ったかは分からない．
共有初期コードは，B と R/F/D の出発点を揃えるための記録である．同じモデル・タスク・seed の組で初期コードが一致することに加え，初期判定や参照先が整合しているかを確認する．コード文字列が同じでも評価方針が異なれば停止判断が変わる可能性があるため，コードの hash だけでは十分でない．評価プロファイルと初期判定を合わせて確認する．
初期失敗だった場合，R/F/D では前回コードを条件別の指示へ組み込み，必要に応じて実行診断を追加する．修正コードを評価した後，L3 へ到達していれば停止する．失敗が続き，最大 3 回の修正へ達していなければ，その round のコードと判定から次の要求を作る．したがって，後続の要求は初期失敗分類だけではなく，途中の出力と評価へ依存する逐次的な手続きである．
この逐次性により，最終 round だけを比較すると，どの情報が与えられていたかや，どの失敗段階を経由したかを見落とす．本研究は round の順序，コード hash，評価分類，履歴の前半・後半の出典を保持して追跡する．最終成功率と履歴分析は代替関係ではなく，異なる粒度で同じ経路を説明するための分析となる．履歴の途中で失敗分類が変わった場合にも，初期類型別の分母は変更しない．

3.2.6 履歴再構成における分岐と到達性

評価器の修正を考える説明用の例として，保存時に失敗と判断されたコードが，新評価では成功になる場合を考える．この場合，新評価を使う手続きではその時点で停止するため，保存されていた次 round のコードは現在の手続きでは生成されない．後続コードを残して都合のよい最終判定を採用すると，成功時停止という設計と矛盾する．再採点後に成功へ変わることは，後続履歴をそのまま保持してよいことを意味しない．
逆に，保存時の成功が新評価では失敗になる場合，現行手続きならさらに修正すべきところで保存履歴が終了している可能性がある．その後のコードは保存されていないため，再採点だけでは補えない．また，成功・失敗の別が変わらなくても，診断文が変われば F/D の次の要求が変わる．診断を後続生成へ与える条件では，採点変更は最終判定だけでなく後続の要求にも影響する．
本研究で分析する到達履歴は，そのような分岐の影響を受けない保存済みの生成履歴の前半部分を利用し，必要な範囲に部分追試による後半部分を接続したものである．後半部分を接続する際には，直前のコードや判定，出典が対応することを確認する．接続先の根拠を確認せずに異なる履歴の一部を接続すると，実際には到達できない経路を作るおそれがある．到達可能性の確認は，保存データを多く残すことより優先される．
部分追試 293 ケースという数と，部分追試で生成した後半履歴 647 ラウンドという数は単位が異なる．一つの追試ケースに複数 round が生じるため，両者の大小を記録の欠落や重複と解釈しない．同様に，現在の到達 round9,249 は，本体の 5,000 条件記録と対応するが同じ母数ではない．本研究ではどの単位の件数なのかをラベルと manifest へ残し，論文の率の分母に誤用しない．
この方法は，すべての過去生成を一つの条件へ完全に揃え直す操作ではない．保存済みの生成履歴の前半部分の生成時期や要求ログの完全性，元 schema の違いは残る．したがって，再構成した生成履歴の比較は現在採用した経路に関する記述として報告する．保存資産を利用して評価の整合性を回復したことと，全条件を同一時期に新生成した比較実験を行ったことを分けて扱う．

3.2.7 記録の検証と失敗の切り分け

記録検証では，予定されたモデル・タスク・seed・条件の組合せを先に定め，実際の記録と照合する．記録総数が予定件数と同じでも，一部が重複し別の一部が欠けている可能性がある．そのた

め，件数の一致に加えてキーの集合差と重複を検査する．予定外記録も別に数え，単に総件数へ足して完了率を高く見せない．
共有初期の検証では，同じ初期組に属する B と R/F/D の初期コードや評価が一致していることを確認する．成功時停止の検証では，成功した後に追加生成を行っていないか，最大修正回数を超えていないかを確認する．また，最終評価が実際の到達 round と一致するかを照合する．これらはモデルの成績とは独立したハーネスの整合性検査であり，失敗率が高いこと自体を検証不合格とするものではない．
生成コードのエラーと API 接続障害は異なる集計対象である．生成したコードの import が失敗した場合は，モデル出力が実行環境に適合しなかった結果として含める．一方，要求を送れなかった場合や応答が得られなかった場合には，その対象についてコード品質を観測できていない．両者を同じ失敗として合算すると，生成の可用性とコードの正しさを混同する．
失敗の分類では，原因を確認して接続障害と生成コードの誤りを区別する．保存時に unknown_error と記録された初期失敗 1 組について，事後確認により未定義変数 sim による NameError と判明したため，本稿では「実行時エラー（未定義変数）」と表記する．保存データの分類キーは変更していない．B4 追加 round にも未定義変数による実行時エラーが 2 件あり，いずれもコード側の NameError である．分類ラベルだけでなく例外の由来を確認し，コード側の失敗として集計した．
公開用の検証資料には，出典ファイル名，保存設定，検証状態，hash，集計表を残す一方，生成コード全文や生の API 応答を無条件に含めない．再現性はすべての内部記録を公開することと同義ではない．どの公開集計から論文表を生成できるかを明確にし，詳細記録を参照する際の粒度を区別する．本稿に記載する hash は内容の同一性を確認するための識別子であり，コードの意味的な正しさを保証するものではない．

3.2.8 静的履歴分析の観察単位

片側救済集合を選ぶ際には，まず F/D の最終 L3 判定から集合を定義し，その集合に属する全 84 組を対象とした．都合のよい数例だけを抜き出すと，説明しやすい変更を過大に強調するおそれがあるため，片側集合全体の両条件を追う．ただし，この選択は F/D の不一致に焦点を当てる分析であり，共通成功や双方失敗を含む全 545 組の変更頻度を推定するための標本抽出ではない．
静的変更分類では，import/API，回路構築・インターフェース，量子ゲート等のテキスト，測定・出力取得を注記する．一つの遷移や履歴で複数領域に該当し得るため，カテゴリ件数の合計が対象組数を超えても矛盾ではない．互いに排他的な失敗類型の表と，重複可能な変更領域の表では，合計の読み方が異なることを明示する．
例えば，ある修正で import 文と回路の測定処理が同時に変わり，その後に成功したとしても，どちらの変更が必要だったかを静的差分だけでは分離できない．さらに，ゲート名の出現数が変わったことは，作用順序や引数の変化を十分に表すものではない．同じゲート名の数でも異なる回路になり得るため，文字列特徴を量子回路の意味そのものと解釈しない．
この分析では，救済対象の不一致が生じた初期類型と，各履歴で変更されたコード領域を対応づける．意味的寄与を確かめるには，個別変更を分離した検証や，回路の作用に基づく別の分析が必要になる．静的確認だけでは，修正が成功に至る仕組みは特定できない．

3.3 実験設定と出典

3.3.1 対象モデルと課題

対象は Q4，G4，Q9，C7，G3 の 5 モデルである．正式なモデルタグは表 2 に示す．タグ中の数値を厳密なパラメータ数とみなさず，モデル識別子として扱う．各モデル 200 初期組であり，10 タスクと seed 0〜19 を含む．
タスクは Bell，GHZ，Deutsch–Jozsa constant/balanced，Bernstein–Vazirani，Grover，QFT，IQFT，QPE，Ansatz の 10 種類である．各タスクは指定入力・期待値を持つ．タスク名が同じでも任意入力を網羅する評価ではない．ハーネスの難度分類を利用するが，普遍的な難しさを表す尺度として解釈しない．

表2 モデルの略号と識別子

略号	モデルタグ	初期組数
Q4	qwen3.5:4b	200
G4	gemma4:e4b-it-qat	200
Q9	qwen3.5:9b	200
C7	qwen2.5-coder:7b	200
G3	gemma3:4b	200

3.3.2 生成条件と元実験の区別

temperature は 0.7，1 要求の生成上限は 1,024 tokens，最大修正回数は 3 回である．保存設定では reasoning_effort は none，timeout_sec は 600，parallel_requests は 1 である．実際に生成された長さは可変であり，上限の同一性は消費量の同一性を意味しない．

元実験は，3 モデル seed 0〜9 の 1,500 条件記録，5 モデル seed 10〜19 の 2,500 条件記録，残り 2 モデル seed 0〜9 の 1,000 条件記録に分かれる．schema や診断文処理の差を保持したうえで，初期組と条件を対応づける．元実験 ID と保存設定の参照先は付録（8.1 節）に示す．

旧 120 件は 2 モデル，seed 0〜2，2 条件，L2 終了の既報であり，現行の L3 による 5,000 条件記録へ合算しない．旧評価，保存コードの再採点，部分追試を三つの独立した主結果として並べない．主集計には元の到達履歴と B4 追加実験を測定禁止条件で確認し，B4 の必要な 1 経路を継続した 2026 年 11 月 4 日の固定スナップショットを用いる [3]．

3.3.3 追加コストと修正回数の集計方針

追加計算コストは共有初期生成を除き，失敗した修正への消費を含めた入力＋出力 tokens の合計である．tokens／救済は総追加 tokens を救済組数で割る．要求時間は保存された elapsed_sec の合計であり，評価時間や全工程の所要時間ではない．

保存履歴の前半の記録値と，部分追試による後半の実測値を分けて保持する．合計は再構成した経路に対する計算値である．金銭単価を測定していないため，金銭費用の削減とは呼ばない．修正上限別の集計も同じ最大 3 修正の観測履歴を区切ったものであり，上限ごとに独立実験した結果ではない．

3.3.4 課題集合の役割と被覆範囲

Bell と GHZ は，多量子ビット間の相関を持つ状態の生成と測定を扱う課題として含める．これらが本ハーネスで易に分類されていても，生成コードが常に正しいとは限らない．API や測定方法の誤りがあれば，算法自体の説明が短い課題でも実行や出力の失敗が起こり得る．難度ラベルは人間にとっての普遍的な難しさやモデルにとっての実際の難しさを測定した尺度ではない．

Deutsch–Jozsa の constant と balanced は，課題条件に応じた期待出力を区別する二つのタスクとして数える．同じ算法名の下でも，条件が異なれば生成コードや評価対象が異なる．したがって，10 タスクという集計では二つを別タスクとして扱う．Bernstein–Vazirani と Grover もそれぞれ指定された対象に対する課題であり，任意の秘密列や任意の探索空間へ一般化した評価ではない．

QFT と IQFT は状態ベクトルを比較する課題として含まれる．分布評価だけでは見えにくい位相の違いを指定入力に対して確認するが，変換全体の同値性検査ではない．QPE は指定した位相推定設定に対する測定分布を確認する．Ansatz も指定された構成と出力を持つ課題であり，任意の変分回路を設計する能力を網羅的に評価しているわけではない．算法名の広い意味と本課題の狭い実装仕様を区別する．

課題集合には，分布と状態の評価方式，量子ビット数，構造制約，使用する回路要素の違いが含まれる．そのため，タスク別成績は単一の難度順へ還元できない．成功率の差には，初期生成でどの種類の誤りが多かったか，出力検査が何を要求するか，修正プロンプトがどの情報を与えるかなど複数の要素が重なる．この分解を十分に行わない段階では，低い救済率を特定の量子算法への理解不足だけで説明しない．

3.3.5 生成設定を揃える意味と限界

同じ temperature を使うことは，各要求のサンプリング設定を揃えるための条件である．しかし，異なるモデルで同じ数値を指定しても，出力の多様性や正解確率が等しくなるとは限らない．モデルの学習内容，語彙，量子化，推論実装などが異なるからである．本研究では temperature の値を設定情報として報告し，モデル間の確率的な挙動が同一であるとは仮定しない．
1,024 tokens という生成上限は，要求ごとの最大出力量を揃える．ただし，実際の出力長は条件と対象に応じて変わる．D では説明・原因分析をコメントとして出力するため，同じ上限内でコード以外の記述が占める割合も変わり得る．このことは比較の条件に含まれるが，今回の集計だけから，出力が長いことが成功を妨げたか，説明が有効だったかを分離して断定することはできない．
timeout と並列要求数は，要求をどのように実行するかの運用条件である．逐次実行であっても，サーバの他の負荷やモデルのロード状態が常に等しいとは限らない．要求時間の比較にはその影響が入り得るため，tokens と時間を同じ意味のコストとして扱わない．時間は保存された要求の経過値であり，評価や待機のすべてを含む工程時間として再解釈しない．
seed の指定は，どの生成をどの初期組へ対応づけるかを再現するための設定として保持する．同じ seed 番号で異なるモデルを呼び出しても，同じ意味の出力候補が得られるわけではない．B4 の追加生成には専用規則があるため，R/F/D と同じ番号を使った同一乱数条件の比較とも呼ばない．本研究の対応は，モデル・タスク・base seed で識別した同じ初期問題を起点とする対応である．

3.3.6 分析対象と固定出典の範囲

本研究の主分析の対象は，保存済みの実験結果と必要な部分追試から再構成した生成履歴，および独立再生成 B4 の追加実験である．B/R/F/D/P には 2026-11-04 修正版の固定集計，B4 には 2026-11-04 修正版の完了集計を用いる．元実験，部分追試，B4 は生成時期と入力情報が異なるため，それぞれの保存設定と由来を区別する．失敗通知と詳細診断を比較する補足実験 R0/R1/R2 は，別の保存設定と出典に基づき 4.8 節に示し，主分析の集計には合算しない．
固定出典を採用する理由は，本文，表，図，考察が同じ集合と判定から導かれるようにするためである．異なる集計時点の数値を混ぜると，過去の評価値と現行の到達履歴を区別できなくなる．本研究では結果表の出典と hash を記録し，本文の数値をそこへ照合する．集計処理を変更した場合には，元の固定ファイルを上書きするのではなく，変更理由と別の出典を明示する．
公開集計には，実験 ID，保存設定，記録の完備性，集計単位を記載する．本文では集計結果とその解釈，検証上の制約を示し，個別記録の由来は manifest と CSV で追跡できるようにする．

4 結果

4.1 全体の結果と比較の前提

表 3 と図 2 に，全 1,000 初期組に対する B・P・R・F・D・B4 の最終 L3 成功率を示す．B は初回生成のみの基準条件，P は生成前注意事項を加えた独立生成である．R/F/D は B の初期失敗コードを修正し，B4 は B の初期コードを共有したうえで，前回コードや診断を与えず再生成する．まず全対象の最終的な到達状況を比較し，次に初期成功の保持を除いて，失敗後の修正・再生成による追加成功を比較する．

表3 6 条件の最終 L3 成功率（全 1,000 初期組）

条件	L3 成功数	分母	最終 L3 成功率
B	455	1,000	45.5%
P	473	1,000	47.3%
R	551	1,000	55.1%
F	638	1,000	63.8%
D	634	1,000	63.4%
B4	671	1,000	67.1%

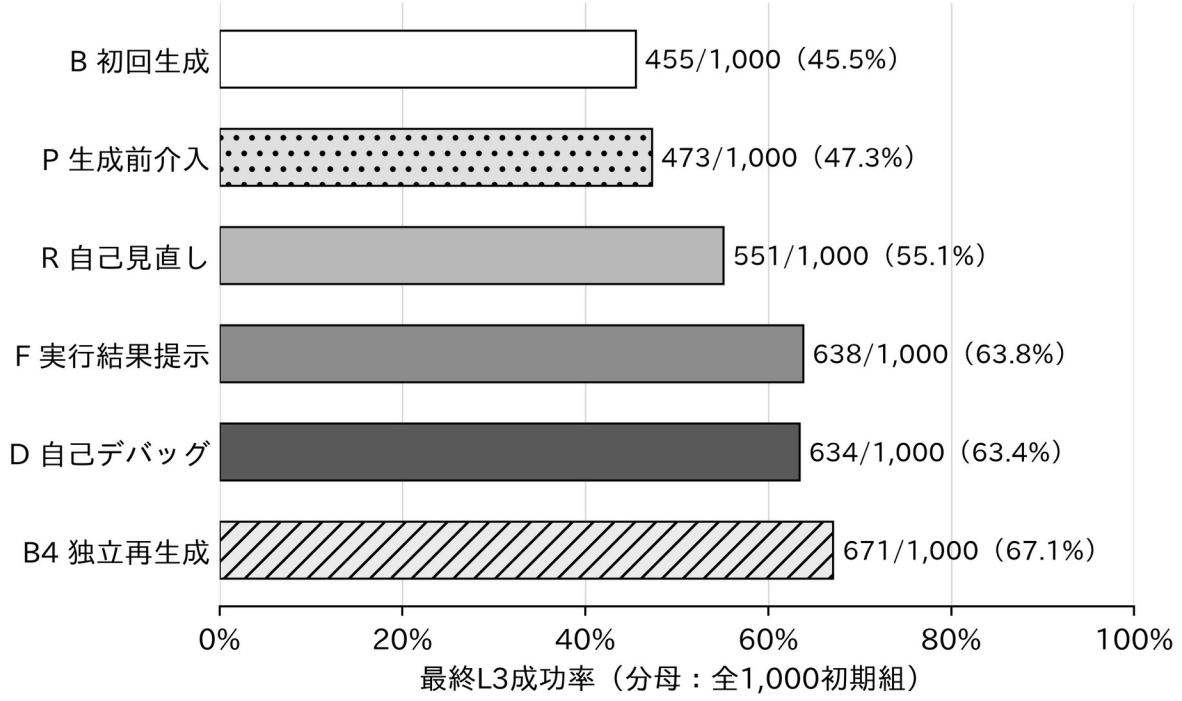


図2 6 条件の最終 L3 成功率（分母は全 1,000 初期組）

1,000 初期組のうち，初期 L3 成功は 455 組，失敗は 545 組であった．表 4 に，この同じ失敗集合に対する R/F/D の救済と B4 の追加成功を示す．F と D はいずれも初期失敗の約 3 分の 1 を救済し，自己見直し R より救済数が多かった．一方，独立再生成 B4 は 216 組が追加成功し，今回の対象では修正条件を上回った．B は追加生成を行わず，P は初期コードを共有しないため，初期失敗への追加成功率の比較には含めない．P の独立生成による改善と悪化は 4.7 節で詳述する．

図 3 は同じ初期失敗 545 組を分母として追加成功率を比較したものである．R/F/D は修正による救済，B4 は独立再生成による追加成功として区別する．

救済率と最終成功率では分母と分子が異なる．例えば F の救済率 33.6%は初期失敗 545 組中 183 組の成功を表し，最終成功率 63.8%は初期成功 455 組の保持を加えた 638/1,000 組を表す．表の追加成功数に初期成功を含めないことで，修正・再生成が新たに成功へ導いた範囲を明確にする．

B4 実験は全 1,000 組の記録が揃い，欠落・重複・予定外記録 0，sanity 検証成功であった．追加 round では未定義変数による実行時エラーが 2 件あり，いずれもコード側の NameError であり，API 接続障害とは区別した．R/F/D は保存履歴と部分追試を接続した再構成結果であり，B4 は別途実施した追加実験である．最大生成機会は初回込み 4 回で揃うが，入力情報・seed 規則・実行時期・出典 schema は一致しない．

表4 初期失敗 545 組からの追加成功

条件	初期失敗数	追加成功数	追加成功率
R	545	96	17.6%
F	545	183	33.6%
D	545	179	32.8%
B4	545	216	39.6%

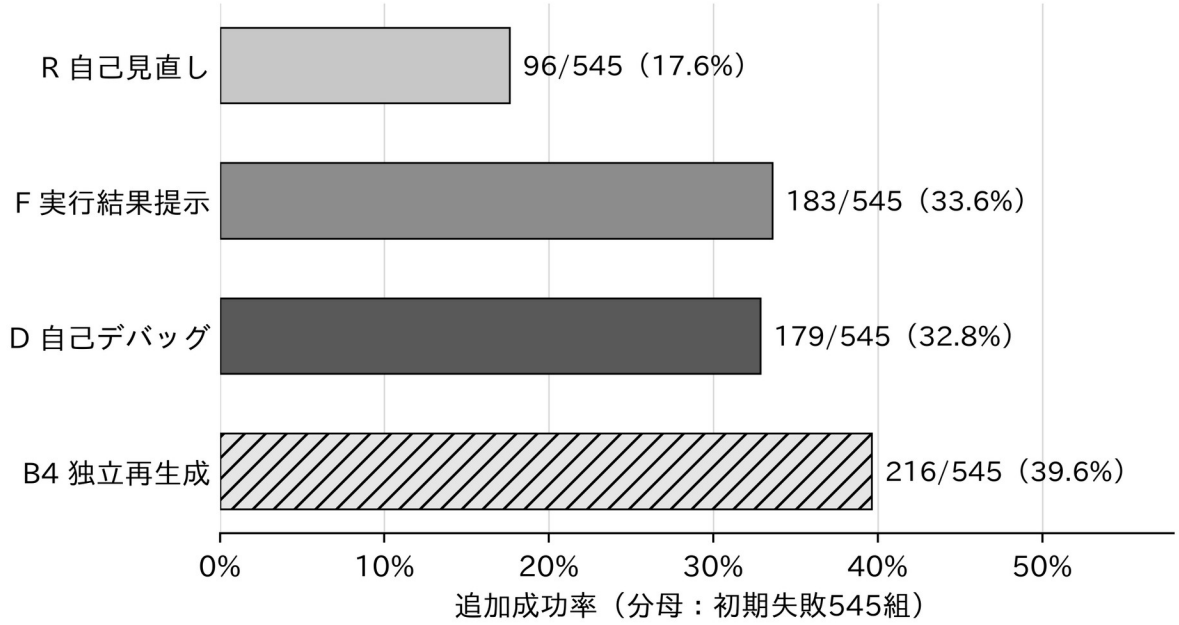


図3 初期失敗 545 組に対する条件別の追加成功率

4.2 モデル別の結果

表 5 は，モデルごとの初期失敗数と追加成功数を示す．G4 では F が 48 組，D が 38 組を救済した一方，Q4 では F78 組，D89 組と D の救済数が多かった．全体で F が 4 組多いという結果は，すべてのモデルで F が多いことを意味しない．C7 では F/D の救済件数が同じだが，成功した初期組が一致するとは限らない．

B4 は G4・C7・Q9 で各修正条件より多く追加成功した一方，Q4 では F/D の救済数を下回った．修正と再生成の成績の関係もモデルで異なる．全体の追加成功率が最も高い条件でも，各モデルで最も多く成功するとは限らない．ただし，モデルタグ以外の学習内容や量子化などを分離した実験ではないため，特定のモデル特性が差を生んだとは断定しない．

モデル間では初期失敗数と失敗類型の構成が異なる．特に G3 の 190 組は全体分母 545 組の大きな部分を占めるが，このモデルを除いて全体率を再定義することはしない．全体率は表の成功数の合計を失敗数の合計で割った値であり，各モデル率の単純平均ではない．全モデルの数値は表へ集約し，本文では条件間の関係が変わる点を中心に解釈する．

表5 モデル別の追加成功数

対象	分母	R	F	D	B4
Q4	162	31	78	89	66
G4	62	30	48	38	51
Q9	66	15	33	28	37
C7	65	15	23	23	46
G3	190	5	1	1	16

4.3 初期失敗類型別の結果

表 6 では，失敗類型を介入前の B で固定している．import 失敗では D の救済数が F より多く，回路構築失敗では F が D を上回った．F/D の率は，import 失敗で 21.4%／26.6%，回路構築失敗で 60.3%／54.0%である．初期類型により条件間の関係が異なることは，介入カタログへ対象類型と分母を記載する根拠になる．ただし，類型はモデル・タスクと関連するため，類型だけによる選択規則の有効性は未検証である．

出力不一致では，B4 の追加成功 46 組が R/F/D を上回った．この分類は出力を評価する段階まで進んだことを表すが，量子算法の論理だけが誤っていたと認定するものではない．測定や出力取得の処理も判定へ関係する．また，構文・import・構築という分類は検査で露見した段階を示すもので，コード内のすべての欠陥を網羅した原因一覧ではない．

表の「実行時（未定義変数）」は，保存時に unknown_error と記録された初期失敗 1 組について，事後確認で判明した未定義変数 sim による NameError を示す本稿上の表記である．保存データの分類キーや件数は変更していない．IF 不適合 11 組，量子ビット数不適合 3 組，実行時エラー 1 組は少数例であり，率だけで一般的な選択規則を導かない．途中の修正で別の類型へ移った組も，初期類型の分母に留める．

表6 初期失敗類型別の追加成功数

対象	分母	R	F	D	B4
import 失敗	271	24	58	72	66
回路構築失敗	126	32	76	68	77
構文誤り	68	21	25	21	24
出力不一致	65	16	24	17	46
IF 不適合	11	1	0	1	2
量子ビット数	3	2	0	0	0
実行時（未定義変数）	1	0	0	0	1

4.4 タスク・難度別の結果

全 10 タスクの分母と追加成功数は表 7 にまとめる．Bell や GHZ では F/D が近い一方，Grover では F が D より多く，QFT では両者の救済数が一致した．この一致は個々の成功対象の一致を保証しない．QPE は初期失敗 94 組に対し F13 組，D11 組の救済であり，初期失敗が多いことと，その失敗を修正しやすいことは別の性質である．

難度別の F/D の救済は，易 22/69 組・25/69 組，中 79/225 組・79/225 組，難 82/251 組・76/251 組であった．難度はハーネスの事前区分であり，普遍的な難しさの尺度ではない．難の中には分布評価と状態評価の課題が混在するため，区分間の差を一つの認知的難度へ帰属しない．

各タスクの初期組数は同じでも失敗数は異なるため，全体率には失敗の多いタスクが大きく寄与する．実際の適用先でタスク構成が違う場合，合算率をそのまま期待成功率として用いることはできない．モデル×タスク，元実験，seed 別の内訳は固定 CSV に保持し，個別の数値の反復列挙に代える．

表7 タスク別の追加成功数

対象	分母	R	F	D	B4
Bell	34	2	10	12	13
GHZ	35	9	12	13	14
DJ constant	42	6	12	16	15
DJ balanced	48	7	25	24	19
BV	62	13	19	24	31
Grover	73	14	23	14	36
QFT	67	21	34	34	30
IQFT	50	3	17	15	22
QPE	94	10	13	11	12
Ansatz	40	11	18	16	24

4.5 成功対象の重なりと履歴分析

表 8 に F/D の対応を示す．共通救済 139 組に対し，F のみ 44 組，D のみ 40 組であり，片側救済は合計 84 組であった．救済数の差は 4 組だが，片側のみ救済された対象は 84 組あった．初期 import 失敗では F のみ 8 組・D のみ 22 組，回路構築失敗では 19 組・11 組となり，周辺の成功率だけでは分からない対象の違いが認められた．
図 4 はこの対応を件数に比例する帯で示したものである．共通救済と片側のみの救済を，双方未救済の集合と合わせて比較できる．
片側 84 組の両条件 168 履歴・404 修正遷移を静的に確認した．救済側の履歴で import/API に関係する変更は F のみ集合の 29 組・D のみ集合の 34 組，回路構築・IF は 25 組・32 組，量子ゲート等のテキストは 39 組・32 組，測定・出力取得は 12 組・16 組に認められた．これらは重複可能な変更領域の注記である．初期失敗の排他的な分類とは異なり，合計を 84 組へ合わせる集計ではない．
B4 と F の共通成功は 130 組，B4 のみ 86 組，F のみ 53 組であった．B4 と D では共通 118 組，B4 のみ 98 組，D のみ 61 組だった．B4 の総成功数が多くても，修正条件の成功集合を包含しない．
三条件のすべてで成功した対象は 99 組，いずれも成功しなかった対象は 255 組であり，少なくとも一条件で成功した集合は 290 組となる．詳細な八分割は固定 overlap.csv へ残した．
F/D の和集合 223 組や，B4 を含む和集合 290 組は，結果を見て定まる事後的な集合である．実行前の選択方策の成功数とは扱わない．
F のみ成功した保存事例として，G4 の IQFT 課題（T7，seed 14）を示す．共有初期コードは QFT を qiskit.circuit から読み込もうとし，保存判定は import 失敗であった．F の 1 回目の修正では読み込み先が qiskit.circuit.library へ変更され，L3 成功となった．同時に，回路構築関数の外にあった使用例コードも削除された．
修正前（F の初期コードから抜粋）
from qiskit.circuit import QFT
修正後（F の round 1 から抜粋）
from qiskit.circuit.library import QFT
同じ初期コードからの D では，説明コメントや QFT 呼び出しの引数を変更したが，読み込み先は qiskit.circuit のまま残った．判定は round 0 から 3 まで import 失敗であった．この事例では，説明内で修正すべき箇所へ言及したことと，実際のコードが変更されたことを区別して確認できる．

D のみ成功した保存事例は，G4 の QPE 課題（T8，seed 10）である．初期コードは qc.inverse_qft(...)を呼び出し，保存判定は回路構築失敗であった．D の 1 回目の修正では QFT を qiskit.circuit.library から読み込み，inverse=True で構成した回路を append する形へ変更され，L3 成功となった．

修正前（D の初期コードから抜粋）

```
qc.inverse_qft(range(n_counting))
```

修正後（D の round 1 から抜粋）

```
from qiskit.circuit.library import QFT
iqft = QFT(n_counting, inverse=True)
qc.append(iqft.to_gate(), range(n_counting))
```

同じ初期コードからの F も逆 QFT を組み立てる形へ変更したが，QFT の読み込み先を qiskit.circuit としたため，round 1 から 3 は import 失敗となった．D でも import 以外のコードやコメントが同時に変化している．掲載した抜粋は変更箇所の一部であり，完全なプログラムではない．

2 事例は，API の利用箇所と保存判定の推移を説明するため，同一モデルから選んだ．頻度や代表性を推定する標本ではなく，片側救済全 84 組の集計を置き換えない．記録キー，各 round のコード hash，判定と由来を case_studies.json に保持した．成功に至った変更と未成功の変更を観察した結果であり，単独の変更や説明指示が成功原因だったとは断定しない．

表8 F/D の救済対象（初期失敗 545 組）

共通	F のみ	D のみ	双方未救済
139	44	40	322

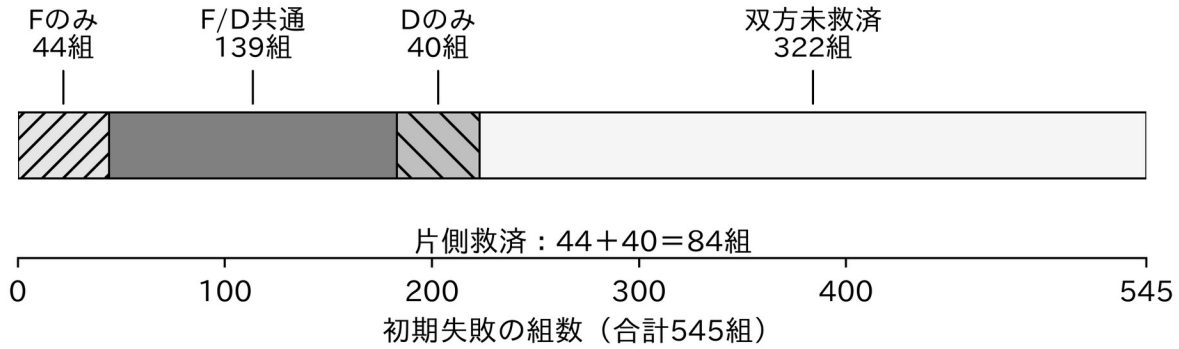


図4 F と D の救済対象の重なり（初期失敗 545 組を排他的な 4 集合に分割）

4.6 追加計算コストと停止回数

表 9 では，共有初期生成を除いた追加要求数・総 tokens・追加成功あたり tokens を比較する．未成功例への消費も含めるのは，実行前にはどの対象が成功するか分からず，失敗した要求も負担するためである．成功履歴だけを抽出した平均と，本指標は異なる．B4 の総追加 996,223 tokens は 1,343 要求で生じたものである．

修正条件の総追加 tokens は，R1,809,845，F1,708,086，D1,975,782 であった．このうち未救済例への消費は，R1,618,684，F1,374,240，D1,610,227 tokens を占めた．F/D には保存済みの生成履歴の前半部分と追試による後半部分が混在し，内訳は固定 CSV に残している．前回コードや診断の有無で入力長も変わるため，表の差を同一条件での診断削除による節約量と解釈しない．要求時間は保存された生成要求の経過時間であり，評価・全工程時間・金銭費用とは区別する．

表 10 に 3 回目の追加救済と消費を示す．F では 2 回後に未救済だった 390 組から 28 組，D では 398 組から 32 組が追加救済された．同じ履歴を 2 回で打ち切る計算では，最終救済のうち F15.3%，D17.9%が失われる．ここで，残存対象に対する成功率と，最終救済に対する損失率では分母が異なる．

後半 round にはそれ以前に失敗し続けた対象だけが残り，初期の同じ集合へ各 round を独立に適用した実験ではない．この比較は同じ観測履歴の打切り計算であり，上限別に独立実験した結果ではない．停止回数の一般的な最適値は確定せず，得られる成功と未成功への追加計算コストを合わせて方策を検討する材料とする．

表9 共有初期を除く追加計算コスト（未成功への消費を含む）

条件	追加要求	追加 tokens	tokens／追加成功
R	1501	1,809,845	18,852.6
F	1368	1,708,086	9,333.8
D	1380	1,975,782	11,037.9
B4	1343	996,223	4,612.1

表10 3 回目の追加救済と消費

条件	追加／未救済	追加 tokens	要求秒	2 回で失う割合
R	19/468	559,141	24,812.3	19.8%
F	28/390	491,676	21,765.5	15.3%
D	32/398	588,970	28,117.5	17.9%

4.7 P の独立生成としての比較

P では，B の初期失敗に対応する成功 124/545 組（22.8%）と，初期成功に対応する失敗 106/455 組（23.3%）があり，最終成功は 473/1,000 組であった．B との差 18 組は改善と悪化の差引きであり，同じコードを修正した結果ではない．初期成功を保持する R/F/D/B4 と同じ分解で解釈しない．

P の 1,000 独立生成の消費は 764,083 tokens である．これは共有初期を除く追加計算コストを示す表 9 とは集計範囲が異なる．生成前注意事項を含めるという操作の成績として保持し，修正介入の救済率や資源効率と無条件に順位づけない．

4.8 補足実験: 失敗通知と詳細診断の比較

失敗の通知と詳細診断の追加を分けて調べる補足実験として，既存の初期失敗 545 組を用い，自己点検・修正の指示を共通にした 3 条件を比較した．R0 は従来の R から外部結果を与えない旨の一文を除き，R1 は R0 に失敗通知だけを，R2 は R1 に検証器の詳細診断を加えた．5 モデル・10 タスクを対象とし，temperature 0.7，出力上限 1,024 tokens で，各条件に追加 1 回の生成を行った．表 11 に主評価の L3 成功数と率を示す．全 1,635 要求を記録し，保存パケット照合で欠落・重複・API 障害は 0 件であった．固定出典は公開 commit 9128f4b の docs/analysis/notification_ablation_20261105T164644282604Z/であり，実験計画，保存設定への参照，検証記録と最大 3 回の副評価を同フォルダに保存した．

表11 補足実験の追加 1 回による L3 成功（初期失敗 545 組）

条件	追加する結果情報	成功数	分母	成功率
R0	結果情報なし	53	545	9.72%
R1	失敗通知	59	545	10.83%
R2	失敗通知と詳細診断	98	545	17.98%

5 考察

5.1 修正と独立再生成の比較から得られる知見

RQ1 への回答．初期失敗 545 組に対する追加成功数は R96 組，F183 組，D179 組，B4 216 組であり，本対象では B4 が最多であった．このため，修正条件を単回生成だけと比べて，診断や前回コードの再利用が成功差の原因だったと結論することはできない．生成機会を増やすこと自体を対照に含める必要がある．一方，B4 と修正条件には片側だけの成功が残り，最大の全体率を持つ条件が他条件を完全に置き換えるともいえない．
前回コードの再利用は，正しい部分を残す一方で誤った構造を引き継ぎ，入力を長くする可能性もある．これらの影響は本研究では分離して検証していない．
RQ2 への回答．F183 組と D179 組の救済数は近かったが，共通救済は 139 組，F のみ 44 組，D のみ 40 組であり，救済対象は一致しなかった．B4 が失敗し，F または D が成功する対象もあるため，全体の追加成功数が最多の条件でも，他条件の成功対象をすべて含むわけではない．
さらに，モデル別・初期失敗類型別に見ると，追加成功数が最多となる条件は一律ではなかった．表 5 では Q4 で D が 89 組と最多であり，他の 4 モデルでは B4 が最多であった．表 6 では import 失敗で D が 72 組，回路構築失敗と出力不一致では B4 がそれぞれ 77 組，46 組と最多であった．この層別の違いと成功対象の不一致を合わせると，全体で最多の B4 だけでは捉えられない修正条件の成功対象がある．本研究の知見は，全体成績，成功集合，失敗の特徴を対応づけて，修正と再生成の選択を検討する根拠を示した点にある．ただし，これらの観測から定めた選択方策が実際に成功数や資源効率を改善するかは未検証である．
共有初期と最大生成機会を揃えても，投入情報，tokens，seed 規則，実行時期は一致しない．成功率の差を介入の因果効果と解釈しない．本研究の到達点は対象集合内の成績と成功対象の範囲の記述であり，一般的なモデル順位や未知タスクでの最適介入は確定していない．
4.8 節の補足実験では，追加 1 回の成功率は失敗通知の追加で 1.10 ポイント，さらに詳細診断を加えると 7.16 ポイント高かった．この対象では，失敗したという通知だけより，詳細診断を含む条件で成功率の改善が大きかった．ただし，同じ 545 組を再利用した観測であり，独立な追加標本ではない．従来の R/F/D とは指示文が異なるため主結果と合算せず，診断による入力長の増加も含む比較として扱う．診断内容だけの効果や，モデルの内的な失敗理解，未知課題での有効性までは認定しない．

5.2 介入カタログの構成と適用範囲

表 12 に，介入カタログに収録した import 失敗，回路構築失敗，出力不一致の 3 類型の観測率を示す．読込み，回路構築，出力評価という異なる失敗段階を比較するため，この 3 類型を取り上げる．件数上位 3 類型という意味ではなく，表 6 の構文誤り 68 組は出力不一致 65 組より多い．率は表 6 と同じ類型の初期失敗数を分母とし，F/D の救済数または B4 の追加成功数を割って算出した．条件間で初期対象を変えていない．

表12 介入カタログ（収録 3 類型の観測率）

初期失敗類型	初期失敗数	F 救済率	D 救済率	B4 追加成功率
import 失敗	271	21.4%	26.6%	24.4%
回路構築失敗	126	60.3%	54.0%	61.1%
出力不一致	65	36.9%	26.2%	70.8%

この一覧は，本研究のモデル・タスク・生成上限と評価条件で観測した比較結果であり，最も高い率の条件を無条件に推奨する規則ではない．F/D は同じ失敗コードを修正し，B4 は課題仕様から独立再生成する．比較条件の違いは 5.1 節に述べたとおりであり，未知の課題での優劣はこの表から判断できない．全体の追加計算コストは表 9 に示したが，類型別の率と全体のコストを組み合わせて類型別の費用対効果を推定しない．
本研究で作成した介入カタログは，初期失敗類型ごとに，対象数と F/D/B4 の成功数・率を対応づけた観測結果の一覧である．intervention_catalog.csv は 3 類型を収録し，各行に分類キー，表示

名，初期失敗数，F/D/B4 の成功数と率の 9 項目を持つ．表 12 はこの CSV の全 3 行について，表示名，初期失敗数，F/D の救済率と B4 の追加成功率を示したものである．全 7 類型の分母と R/F/D/B4 の追加成功数は表 6 に示す．
カタログの適用条件と出典は，第 3 章と付録（8.1 節）に示した保存設定・manifest・検証記録を併読する．モデル・課題・評価方式，利用する入力情報，生成上限・停止条件は共通の実験条件として記載し，類型別の CSV には重複して収録していない．失敗した生成要求の消費も含む追加計算コストは表 9 の全体集計であり，類型別の値をこの CSV へ収録したものではない．同じ import 失敗でも，SDK の版や利用可能な API が違えば本研究の結果をそのまま適用できないため，観測率とその適用範囲を結び付けて読む必要がある．
import 失敗と回路構築失敗では F と D の救済数の関係が異なった．この観測は表 6 の初期失敗数と救済数に対応する．今後の選択方策では，失敗類型に加え，診断が示す API 名や操作箇所，前回コードをどこまで残すかが検討対象になる．ただし，診断は潜在的な欠陥のすべてを示すものではない．「この集合では D が多かった」という観測と，「今後は必ず D を選ぶ」という方策の有効性を混同しない．
出力不一致の観測率を解釈する際には，第 3 章に示した分布評価と状態評価の違い，指定入力，距離閾値，構造要件を確認する．出力が違うというラベルだけで，量子算法の論理，測定順序，出力取得の問題を一括して原因認定しない．IF 不適合など少数類型の件数は表 6 に残すが，高率や低率だけから未知の対象への選択規則を導かない．
B4 は修正と別の対照条件として比較する．第 3 章では，課題仕様から再生成すること，前回コード・診断を渡さないこと，専用 seed と成功時停止を明示した．単に「再生成」とだけ書くと，一括候補生成や任意回数の再試行と区別できない．同じ操作名でも予算や停止が異なれば，成功集合と消費は変わり得る．
成功率・追加計算コスト・対象の被覆では，最良となる条件が異なり得る．カタログはこれらを比較する根拠を提供する形式であり，今回実証済みの自動選択器ではない．F/D の和集合を利用するにも，事前選択の規則か両条件を実行する手順が必要である．その追加計算コストと停止を含む評価は，今後の方策検証として扱う．

5.3 資源制約と停止方策への含意

RQ3 への回答．未救済例への消費を含む追加 tokens／救済の計算値は R18,852.6，F9,333.8，D11,037.9 であった．独立再生成 B4 の追加 tokens／追加成功は 4,612.1 であり，未成功例への消費を含む総追加 996,223 tokens を追加成功 216 組で割った値である．3 回目の修正では F28 組，D32 組が追加救済された．追加計算コストと救済数は条件ごとに異なり，修正回数を減らす際には失われる救済も考慮する必要がある．
要求回数の上限を揃える比較と，tokens 予算を揃える比較は異なる．前回コードや診断を含む条件は入力が長くなり得るため，同じ回数の要求を許しても資源配分は一致しない．本研究は生成機会の上限を対応づけた比較であり，同じ tokens 予算下の最適方策を直接測定したものではない．
未成功例への消費を含めることで，成功した履歴だけを観察する場合には見えない負担を記録できた．ただし，未成功例への消費が大きくても，その対象を事前に識別できるとは限らない．停止方策には，実行前または途中で利用可能な特徴を用いる必要があり，最終結果を見た後の対象選別と区別する．
3 回目の寄与は，救済を失う割合と追加計算コストを同時に検討する材料になる．しかし，最初から短い上限を課す実験では出力や利用者の行動が変わり得る．失敗類型が変わらない場合の停止や，残予算に応じた停止を評価するには，その規則の下で到達可能な逐次経路を実際に生成する必要がある．保存履歴の接続だけで，新しい方策の成績を確定することはできない．
実運用で待機時間，tokens，成功数のどれを重視するかによっても，選択する条件は変わる．本研究は利用者の読解時間，電力，金銭単価，評価を含む全工程時間を測定していない．記録上の tokens 差を総作業量や金銭費用の同率の削減と解釈せず，測定した資源の範囲に限定して報告する．

5.4 妥当性と限界

P の注意事項には，量子ビット数と古典ビット数を一致させる指示が含まれる．一方，共通 system メッセージでは build_circuit()内に測定操作を追加しないよう指示しており，課題によっては古典ビットを必要としない．両者は必ずしも論理矛盾ではないが，P の一律の注意事項が課題仕様や共通指示と競合し，不要な古典ビットの追加を促す可能性がある．この指示が生成コードや P の成績に与える影響は分離して検証していない．したがって，P の結果は付録 8.3 に示す注意事項の構成に依存する観測として扱い，予防的注意事項一般の有効性へは一般化しない．
内的妥当性に関して，共有初期は初回生成の違いを抑えるが，入力長や指示，実行時期などの交絡は残る．初期性能が異なるモデルで残る失敗の構成も同じではない．R/F/D には保存済みの生成履歴の前半部分と追試による後半部分が混在するため，一つの同時実験だったと表現しない．また，成功時停止後の品質変化は観測しておらず，退行 0 を自由な編集を続けても正しさを保つ保証と扱わない．
構成概念妥当性に関して，L3 は指定入力の出力と構造要件への適合である．任意入力の同値性，保守性，読みやすさ，実機での品質など，実用上の性質すべてを表す指標ではない．命令数の範囲も意味的な正しさを完全には特徴づけない．評価範囲を拡張する場合，保存コードの再採点と，変更後の診断による後続生成を区別する必要がある．
静的履歴分析は文字列上の変更領域を観察するもので，量子変換の意味的差を測定していない．ゲート名の増減から量子論理の訂正を認定できず，複数同時変更の寄与も分離できない．片側 84 組を全件確認したことは，その集合内で都合のよい事例だけを選ばなかったことを意味するが，全 545 組の変更頻度を推定する無作為標本ではない．
外的妥当性に関して，5 モデル・10 タスクの build_circuit()生成は量子開発全体を代表しない．複数ファイル，大規模回路，ツール検索，実機ノイズなどは対象外である．タグ中の数値からパラメータ数や量子化の効果を分離して論じず，モデル・課題・生成上限・実行環境を含む観測として解釈する．有限 shots と測定 seed への依存も残る．
統計的解釈に関して，同じ初期組の複数条件，同じタスクの複数 seed には依存がある．条件記録や round を独立標本として数えない．本研究は件数，率，集合，消費の探索的な記述であり，有意差を検定していないことを「差がないと証明した」と言い換えない．多くの層別集計から選んだ規則を同じデータで評価すると楽観的になるため，探索と検証の集合を分ける必要がある．

5.5 評価漏れの修正範囲と残る制約

測定違反 4 組のうち 1 組は現行の到達履歴に採用していない保存 D コードであり，主結果へ影響しない．現在の経路では D の Grover・Q4・seed 4・round 3 と，B4 の DJ constant・C7・seed 3・round 1 および seed 5・round 3 の 3 経路へ影響した．最終 round 3 で不合格となった 2 経路は生成上限に達している．残る B4 の 1 経路だけ後続生成が必要となったため，保存時と同じモデル digest・生成設定・seed 規則で継続した．round 2 の追加 1 要求，408 tokens で L3 成功し，round 3 は生成していない．
旧集計の D635 組・B4 672 組に対し，測定禁止修正と上記継続後の最終成功は D634 組・B4 671 組となった．旧固定集計と生成履歴を保持し，修正版は 2026-11-04 の別スナップショットへ保存した．後続生成の未完了経路は 0 である．これは全条件を新評価で初回から再生成した比較ではなく，後日実行した 1 要求が含まれる．同じモデル digest と seed でも実行時期やサーバ環境の完全な同一性を保証しない．
主 L3 は回路内測定禁止を含むが，任意入力の算法同値性や全指示の完全な遵守を保証しない．厳格評価の成功数を主 L3 へ混ぜず，評価をさらに拡張する場合には，保存コードの再採点，停止・診断の変化，後続生成の要否を改めて分けて確認する．

5.6 今後の検証計画

第一に，介入選択に使う特徴を初期失敗分類やモデル，出力評価方式など介入選択時に得られる情報へ限定し，規則を事前固定する．候補規則を作る集合と検証する集合を分け，新しい seed・

タスクで成功数と総追加計算コストを測定する．事後的な和集合を方策の成績とする比較から進むための計画である．
第二に，要求数上限だけでなく，入力と出力を含む tokens 予算を揃えた比較を行う．失敗要求，再送，評価時間の扱いを先に定め，結果を見て有利な予算だけを主結果に選ばない．F/D の比較では診断を合わせ，説明・原因分析の指示，出力コメント，入力長と生成上限を明示し，何を変えた実験なのかを限定する．
第三に，固定回数，失敗分類の停滞，残予算などに基づく停止規則を比較する．各規則で到達履歴を生成し，停止で失った成功と節約した消費を同じ対象で測定する．保存履歴の打切り計算は設計の材料に用いるが，実際の方策検証の代わりにはしない．
第四に，測定 seed，距離閾値，入力集合，生成上限を変え，成功集合と条件間の対応の安定性を確認する．評価変更が停止や診断に及ぶ範囲を監査し，再採点の感度分析と新評価下の生成実験を区別する．これらは未実施の検証計画である．

6 おわりに

6.1 研究成果

本研究は，共通 system の回路内測定禁止を補った主 L3 に基づき，修正型介入と独立再生成を追加成功数，成功対象，追加計算コストで比較した．別の厳格評価は成功数へ含めていない．第一に，初回込み最大 4 回という生成機会上限を揃えた比較では，初期失敗 545 組に対する追加成功数は B4 が 216 組と最多であり，R96 組，F183 組，D179 組を上回った．全 1,000 初期組の最終 L3 成功率は B45.5%，P47.3%，R55.1%，F63.8%，D63.4%，B4 67.1%であった．
第二に，F/D の救済数の差は 4 組だが，共通救済 139 組，F のみ 44 組，D のみ 40 組であり，救済対象は一致しなかった．全片側履歴 84 組を静的に追跡した一方，変更の意味的寄与の分離は今後に残した．第三に，モデル別・初期失敗類型別では，追加成功数が最多となる条件が変わった．Q4 と import 失敗では D が最多であり，他の 4 モデル，回路構築失敗，出力不一致では B4 が最多であった．全体成績の高さだけでなく，成功対象の違いと失敗の特徴を併せて比較することが，本研究で得た知見である．これらは修正と再生成の選択を検討する根拠となるが，選択方策による性能改善は未検証である．
主要な初期失敗 3 類型（import 失敗，回路構築失敗，出力不一致）について，初期失敗数と F/D による修正の救済率，B4 による独立再生成の追加成功率を比較する介入カタログを作成した．成果物 intervention_catalog.csv には 3 類型の成功数と率を収録し，表 12 にその観測率を示した．カタログは本実験の観測結果を整理したものであり，自動選択方策の有効性を実証したものではない．最大生成機会上限を揃えても入力情報・seed 規則・実行時期は一致しないため，観測差を介入の因果効果や未知タスクへの推奨規則として扱わない．

6.2 今後の課題

次の段階では，介入選択時や停止判断時に利用できる特徴に基づいて規則を事前固定し，別の seed・タスクで検証する．要求数と tokens 予算，出典時期，評価範囲を明示し，事後的な成功集合と実際に使える方策を区別する．L3 が指定入力に限定されることや，共有初期・タスクの依存を保ったまま，比較の範囲を広げる必要がある．
カタログの役割は，一つの条件を常に推奨することではなく，何を対象に，何を与え，どこで止め，どの検査を通過したかを追跡できる形にすることである．今後は，未成功対象への消費も含む観測結果を基に選択方策を定め，別の対象で検証する必要がある．

7 参考文献

[1] A. Madaan et al.: Self-Refine: Iterative Refinement with Self-Feedback, arXiv:2303.17651v2 (2023). https://arxiv.org/abs/2303.17651v2
[2] X. Chen, M. Lin, N. Schärli, D. Zhou: Teaching Large Language Models to Self-Debug, arXiv:2304.05128v2 (2023). https://arxiv.org/abs/2304.05128v2
[3] 伊藤悠真: 測定禁止修正版の固定集計 (2026-11-04).
固定出典: docs/analysis/measurement_ban_corrected_20261104/.
共通 system の回路内測定禁止を補い, B4 の 1 経路を追加 1 要求で継続した.
旧固定集計・生成履歴は保持した. 参照ファイル: manifest, sanity, 集計 CSV.
固定版: 公開固定版 (72c03f8) (2026 年 11 月 4 日固定).
[4] M. Chen et al.: Evaluating Large Language Models Trained on Code, arXiv:2107.03374v2 (2021). https://arxiv.org/abs/2107.03374v2
[5] J. Austin et al.: Program Synthesis with Large Language Models, arXiv:2108.07732 (2021). https://arxiv.org/abs/2108.07732
[6] J. Huang et al.: Large Language Models Cannot Self-Correct Reasoning Yet, arXiv:2310.01798v2 (2024). https://arxiv.org/abs/2310.01798v2
[7] S. Vishwakarma et al.: Qiskit HumanEval: An Evaluation Benchmark For Quantum Code Generative Models, arXiv:2406.14712 (2024). https://arxiv.org/abs/2406.14712
[8] IBM Quantum: Construct circuits, IBM Quantum Documentation (2026 年 10 月 29 日参照). https://quantum.cloud.ibm.com/docs/en/guides/construct-circuits
[9] Qiskit Development Team: Qiskit Aer documentation (2026 年 10 月 29 日参照). https://qiskit.github.io/qiskit-aer/
文献[1], [2], [4]〜[7]の書誌情報と本文で参照した主張の範囲は, 原論文の公開ページで確認した. 文献[8], [9]は Qiskit と Qiskit Aer の公式資料である. 文献[3]は本研究の一次資料で, 集計時点・保存設定・検証記録を固定している.

8 付録

8.1 出典と再現性

8.1.1 5 モデル追加実験の保存設定

results/shared_five_models_full_20261010T041019864060Z/experiment_config.json
設定 SHA-256：5c9232f76cc6758b52983fe90ec0757ba2ffe2c4972884a25736f09edd45f22a
保存 schema：2.1．ハーネス commit：9c1833eba6020369247358c04e462e0dc1982705．モデル digest とプロンプト関数 hash は元 manifest に保持．

8.1.2 3 モデル本実験の保存設定

results/shared_three_models_full_20261008T103231611899Z/experiment_config.json
設定 SHA-256：a4cd6ab1126e8927aa3a76c4fcf7392026007215d0388d8b6f73c3bba84691ec
保存 schema：2.0．ハーネス commit：9c6f952b41ac661bb42bb1ffe4d3f37282260658．モデル digest とプロンプト関数 hash は元 manifest に保持．

8.1.3 2 モデル補完実験の保存設定

results/shared_two_models_seed0_9_full_20261012T100658266051Z/experiment_config.json
設定 SHA-256：dd6b694be0b0c03b3bc33ecc13c69872104a5dba2b66706389b8163a618b0a9b
保存 schema：2.1．ハーネス commit：7ccef09db9bff16785b08f2e7f1ff542cf64b849．モデル digest とプロンプト関数 hash は元 manifest に保持．

8.1.4 固定集計と公開範囲

固定集計時点（UTC）：2026-11-04T11:49:57.181940+00:00
主出典：docs/analysis/measurement_ban_corrected_20261104/results_snapshot.json
B4 追加実験：independent_4_full_20261020T185811381595Z．固定出典：docs/analysis/measurement_ban_corrected_20261104/．runner_experiment_config.json，runner_manifest.json，summary.csv，pairwise.csv，sanity_report.json，source_manifest.json を参照．
全分母：docs/analysis/measurement_ban_corrected_20261104/core/repair_summary.csv
追加計算コストの保存先：docs/analysis/measurement_ban_corrected_20261104/
参照ファイル：cost_by_cap_and_stratum.csv，third_round_summary.csv
片側救済全履歴：discordant_cases.csv と discordant_transitions.csv
検証：core/sanity_report.json，sanity_report.json，validation_report.json
介入カタログ：intervention_catalog.csv．事例の記録キー・判定・コード hash・短い抜粋：case_studies.json．固定プロンプト：experiment_prompts.json．追加資料の出典と hash：enrichment_manifest.json．
本稿では検証済み事例の短いコード抜粋と固定プロンプトを掲載する．API 応答全文・生成コード全文・生例外文・raw.jsonl・秘密情報は掲載しない．

8.2 指標の読み方

8.2.1 集計単位

初期組はモデル×タスク×seed の 1,000 組，B/R/F/D/P の条件記録は 5,000 記録，到達 round は 9,249 round である．B4 の追加実験 1,000 組は別に集計する．単位を混ぜて成功率の分母にしない．

8.2.2 退行 0 の意味

R/F/D は初期成功を保持し，修正中も成功時に停止する．退行 0 はこの設計を反映しており，自由な修正を続けても正しさが保たれる保証ではない．

8.2.3 静的変更の意味

import/API やゲート名の変更は観察された文字列上の事実であり，その変更単独の因果的効果や意味的正しさを認定したものではない．

8.2.4 コストの意味

共有初期を除く入力＋出力 tokens と生成要求時間を集計した．再構成した生成履歴の計算値で，評価時間・全工程時間・金銭費用とは異なる．

8.3 実験プロンプト

本節には，主分析の元実験に用いた system メッセージ，課題仕様 10 種，修正条件 R/F/D と予防条件 P の固定文を掲載する．B4 の追加生成には B と同じ課題仕様だけを与え，前回コードや診断は渡さない．元 3 実験の保存 commit で固定文が一致することを照合した．コード・診断が入る部分は明示的なプレースホルダーへ置き換えた．4.8 節の補足実験 R0/R1/R2 の固定文は，同節の出典フォルダ内の prompts/に別途保存している．
掲載内容は保存されたソースから復元したテンプレートであり，個別のネットワーク要求全文を直接記録したものではない．課題仕様を〈課題仕様〉へ，直前の生成コードを〈前回コード〉へ，評価から作る診断文を〈直前の評価から生成した診断〉へ対応させる．実際の診断文は round ごとの評価値と当時の処理に依存し，元実験と部分追試で同一とは限らない．出典 commit とファイル hash は enrichment_manifest.json に示す．

8.3.1 共通 system メッセージ

You are an expert Qiskit programmer. Generate a complete, runnable Python script using qiskit (version 2.x) and qiskit-aer.

Strict requirements:
- Import all necessary modules at the top.
- Define exactly one function named `build_circuit() -> QuantumCircuit`.
- The function must return a QuantumCircuit with the requested number of qubits.
- DO NOT add measurement operations inside `build_circuit`. Measurement is added by the evaluator.
- Output ONLY the Python code in a single fenced block: ```python ... ```
- No explanation outside the code block.

8.3.2 課題仕様 T1_Bell

Task: Construct the Bell state (|00⟩ + |11⟩) / √2 on 2 qubits.

Expected behavior:
- Apply H to qubit 0, then CNOT with qubit 0 as control and qubit 1 as target.
- After execution, measuring all qubits should yield "00" with probability 0.5 and "11" with probability 0.5.

Number of qubits: 2

8.3.3 課題仕様 T2_GHZ

Task: Construct the GHZ state (|000⟩ + |111⟩) / √2 on 3 qubits.

Expected behavior:
- Apply H to qubit 0, then CNOT(0, 1), then CNOT(1, 2).
- After execution, measuring all qubits should yield "000" with probability 0.5 and "111" with probability 0.5.

Number of qubits: 3

8.3.4 課題仕様 T3a_DJ_constant

Task: Implement the Deutsch-Jozsa algorithm for the constant function f(x1, x2) = 0 (the all-zero function).

Setup:
- 3 total qubits: q0 and q1 are input qubits, q2 is the ancilla.

- The ancilla q2 must be initialized to |1⟩ (apply X gate before the H gates).
- After Hadamards, apply the oracle for f (identity for constant 0; do nothing).
- Apply Hadamards on the input qubits q0 and q1 only (NOT the ancilla).

Expected: input qubits q0, q1 deterministically measure as "00" (constant detected).
The ancilla measurement is 50/50 between 0 and 1.

Number of qubits: 3

8.3.5 課題仕様 T3b_DJ_balanced

Task: Implement the Deutsch-Jozsa algorithm for the balanced function f(x1, x2) = x1 XOR x2.

Setup:
- 3 total qubits: q0 and q1 are input qubits, q2 is the ancilla.
- Initialize ancilla q2 to |1⟩ (X gate first).
- Apply H to all 3 qubits.
- Oracle for f = x1 XOR x2: CNOT(q0, q2), CNOT(q1, q2).
- Apply final H on input qubits q0 and q1 only.

Expected: input qubits q0, q1 deterministically measure as "11" (balanced detected).
The ancilla measurement is 50/50.

Number of qubits: 3

8.3.6 課題仕様 T4_BV_011

Task: Implement the Bernstein-Vazirani algorithm to recover the hidden bitstring s = "011" (binary).

Bitstring convention: s is read as bits (s[2], s[1], s[0]) where s[i] corresponds to qubit i.
So s[0]=1, s[1]=1, s[2]=0.

Setup:
- 4 total qubits: q0, q1, q2 are the query register, q3 is the ancilla.
- Initialize ancilla q3 to |1⟩ (X gate).
- Apply H to all 4 qubits.
- Oracle for f(x) = s · x = (s[0]·x[0] + s[1]·x[1] + s[2]·x[2]) mod 2:
  - Apply CNOT(q_i, q3) for each i where s[i] = 1.
  - For s = "011": apply CNOT(q0, q3) and CNOT(q1, q3).
- Apply final H on the query qubits q0, q1, q2.

Expected: q0, q1, q2 measure as "011" (= s) with certainty. Ancilla q3 measures 50/50.

Number of qubits: 4

8.3.7 課題仕様 T5_Grover_11

Task: Implement Grover's search algorithm on 2 qubits with the marked state |11⟩, using exactly one Grover iteration.

For n=2 and 1 marked state, the Grover iteration produces the marked state with probability 1 (exact).

Algorithm:

1. Initialize 2 qubits in uniform superposition (H on both).
2. Apply oracle that marks |11⟩ with a phase flip: e.g., CZ(0,1).
3. Apply diffusion operator (inversion about the mean): H on all, X on all, CZ(0,1), X on all, H on all.

Expected: measuring should yield "11" with probability 1.

Number of qubits: 2

8.3.8 課題仕様 T6_QFT_001

Task: Apply the 3-qubit Quantum Fourier Transform (QFT) to the input state |001⟩.

In Qiskit's little-endian convention, |001⟩ means qubit 0 = |1⟩ and qubits 1, 2 = |0⟩.

Algorithm:
1. Initialize the input state by applying X to qubit 0.
2. Apply QFT to all 3 qubits. You may use qiskit.circuit.library.QFT or implement it manually.
   Use the standard convention with swap gates at the end (do_swaps=True).

The expected output statevector is determined by the canonical QFT mapping:
QFT|j⟩ = (1/√N) Σ_k exp(2πi · j · k / N) |k⟩ with N = 2^3 = 8

Number of qubits: 3

8.3.9 課題仕様 T7_IQFT_001

Task: Apply the INVERSE 3-qubit Quantum Fourier Transform (inverse QFT, IQFT) to the input state |001⟩.

In Qiskit's little-endian convention, |001⟩ means qubit 0 = |1⟩ and qubits 1, 2 = |0⟩.

Algorithm:
1. Initialize the input state by applying X to qubit 0.
2. Apply the INVERSE QFT (not the forward QFT) to all 3 qubits.
   You may use qiskit.circuit.library.QFT with inverse=True, or implement it manually.
   Use the standard convention with swap gates (do_swaps=True).

Note: the inverse QFT is the conjugate transpose of the forward QFT; getting the direction
(sign of the phase rotations) and the swap placement right is essential.

Number of qubits: 3

8.3.10 課題仕様 T8_QPE_001

Task: Implement 3-bit Quantum Phase Estimation (QPE) for the single-qubit phase gate P(theta)
with theta = pi/2, acting on its eigenstate |1>.

Setup (4 qubits total):
- Qubits 0, 1, 2: the counting register (3 estimation qubits).
- Qubit 3: the eigenstate register, prepared in |1> (eigenstate of P(theta) with eigenvalue e^{i*theta}).

Algorithm:
1. Apply X to qubit 3 to prepare the eigenstate |1>.

2. Apply H to each of the counting qubits 0, 1, 2.
3. For each counting qubit j (j = 0, 1, 2), apply controlled-P(theta) with qubit j as control and
   qubit 3 as target, repeated 2**j times (equivalently controlled-P(theta * 2**j)).
4. Apply the inverse QFT to the counting register (qubits 0, 1, 2).

Since theta = pi/2 corresponds to phase phi = theta/(2*pi) = 1/4 = 0.010 in binary, the counting register should deterministically encode the estimate. Do NOT add measurements; the evaluator adds them.

Number of qubits: 4

8.3.11 課題仕様 T9_Ansatz_001

Task: Build a 2-qubit hardware-efficient ansatz with the following exact structure and fixed rotation angles (in radians). Apply the operations in this order:

1. RY(pi/4) on qubit 0
2. RY(pi/3) on qubit 1
3. CX with qubit 0 as control and qubit 1 as target
4. RY(pi/6) on qubit 0
5. RY(pi/5) on qubit 1

This is a single-entangling-layer ansatz: a layer of parameterized RY rotations, one CX entangler, then a second layer of RY rotations. Use the angle values exactly as given. Do NOT add measurements.

Number of qubits: 2

8.3.12 修正テンプレート R: 自己見直し

〈課題仕様〉

--- 前回のコード ---
あなたは前回、以下のコードを生成しました。外部のテスト結果や正解は与えません。

```python
〈前回コード〉
```

このコードを自分で慎重に点検してください。タスク要件、Qiskit API、量子回路の意味、build_circuit() の仕様を確認し、問題がある場合は修正してください。
最終的に、完全な修正版 Python コードだけを ```python``` ブロックで出力してください。

8.3.13 修正テンプレート F: 実行結果提示

〈課題仕様〉

--- 前回の試行結果 ---
あなたは以下のコードを生成しましたが、実行ベース検証に失敗しました。

```python
〈前回コード〉
```

検証器から得られた情報:

〈直前の評価から生成した診断〉

この実行フィードバックを使って問題を修正し、完全な Python コードを ```python ``` ブロックで出力してください。

8.3.14 修正テンプレート D: 自己デバッグ

〈課題仕様〉

--- 前回の試行結果 ---
```python
〈前回コード〉
```

検証器から得られた情報:
〈直前の評価から生成した診断〉

次の順序で自己デバッグしてください。
1. 前回コードが何をしているかを説明する。
2. タスク要件と検証結果を照合し、失敗原因を特定する。
3. 原因を踏まえてコードを修正する。
4. system の『コードブロックのみ』制約を守るため、説明と原因分析は 修正版コードブロック先頭の Python コメント（# DEBUG: ...）として 3〜6 行残す。その後に完全な修正版コードを書く。

8.3.15 P の予防的注意事項

P は課題仕様の後に改行を 2 個挿入し，次の固定文を追加して 1 回だけ生成する．B/R/F/D の初回は課題仕様だけを与える．
量子コード生成では以下の点に注意してください。

1. 指定された関数名と戻り値形式を必ず守ること。
2. Qiskit に存在しない API を使用しないこと。
3. 量子ビット数と古典ビット数を一致させること。
4. 測定結果のビット順序に注意すること。
5. 期待される測定分布または状態ベクトルと一致する回路を作ること。
6. 不要なゲートを追加しないこと。
7. 指定されたインターフェースを必ず実装すること。